Basi di Dati

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Basi di Dati" erogato nell'A.A. 2026 dal professore Giuseppe Perelli per il C.d.L Informatica (33503)*

9 novembre 2026

Indice

1 La memorizzazione dei dati nel passato **4**

1.1 Sistema informativo e sistema informatico 4

1.2 La gestione dei dati prima dei DBMS 5

2 Basi di dati e DBMS **6**

2.1 Architettura ANSI/SPARC 9

2.1.1 Schema esterno 9

2.1.2 Schema logico 10

2.1.3 Schema fisico 10

2.2 Le transazioni 11

3 Progettazione di basi di dati **14**

3.1 Modelli informatici 15

3.2 Entità, collezioni e attributi 16

3.3 Le associazioni 17

3.4 Le gerarchie di classi 19

3.4.1 Vincoli su insiemi di sottoclassi 21

3.5 Le fasi per la costruzione di una base di dati 22

4 Progettazione concettuale **23**

4.1 Il modello E-R 23

4.1.1 Entità 24

4.1.2 Associazioni 24

4.1.3 Attributi 26

4.1.4 Cardinalità di associazioni 27

4.1.5 Identificatore di un'entità 28

4.1.6 Generalizzazioni 29

5 Progettazione logica **31**

5.1 Il modello relazionale 31
5.1.1 Chiavi di una relazione 33
5.2 Ristrutturazione del modello concettuale 34
5.2.1 Analisi delle ridondanze 34
5.2.2 Eliminazione delle generalizzazioni 35
5.2.3 Partizionamento/accorpamento di entità e associazioni 36
5.2.4 Scelta degli identificatori primari 38
5.3 La traduzione da schema concettuale a logico 38
5.3.1 Le entità 38
5.3.2 Le associazioni 39
5.3.3 I valori NULL 44
5.4 Vincoli di integrità 45

6 Algebra relazionale 48

6.1 Proiezione 48
6.2 Selezione 50
6.3 Unione 50
6.4 Differenza 53
6.5 Intersezione 54
6.6 Prodotto cartesiano 55
6.7 Ridenominazione 56
6.8 Join naturale 57
6.9 Theta Join 58
6.10 Quantificatori unari e globali 59

7 Il linguaggio SQL 63

7.1 Il linguaggio di definizione dei dati: DDL 64
7.1.1 Domini di dati 64
7.1.2 Creazione di una tabella e constraint 65
7.1.3 Creazione dei vincoli inter-relazionali 66

7.1.4 Modifica degli schemi 70
7.1.5 Creazione delle viste 70
7.1.6 Basi di dati attive e Trigger 71
7.2 Il linguaggio di interrogazione dei dati: DQL 75
7.2.1 I comandi essenziali 76
7.2.2 Interrogazione su più tabelle 80
7.2.3 Ordinamento del risultato 82
7.2.4 Operatori aggregati 82
7.2.5 Operatori insiemistici 88
7.2.6 Interrogazioni nidificate 89
7.3 Il linguaggio di manipolazione dei dati: DML 91
7.3.1 L'inserimento di dati 91
7.3.2 La cancellazione dei dati 93
7.3.3 La modifica dei dati 93
7.4 Il linguaggio per il controllo degli accessi: DCL 94
7.4.1 La concessione dei privilegi 95
7.4.2 La revoca dei privilegi 96
7.4.3 Le transazioni in SQL 96

8 Dipendenze funzionali 99

8.1 Chiusura di un insieme di dipendenze funzionali 104

1

La memorizzazione dei dati nel passato

Si ritiene opportuno, come introduzione al corso, fornire vari esempi e una definizione abbastanza comune di "base di dati". Interagiamo con basi di dati costantemente durante la nostra quotidianità: accedendo all'applicazione della nostra banca, utilizzando i social network, effettuando un acquisto su piattaforme di E-Commerce, ...
Questi sono solo pochi degli innumerevoli esempi di basi di dati con le quali ci troviamo a contatto ogni giorno. Possiamo dunque concludere che una definizione molto poco formale e spicciola di "base di dati" sia letteralmente *un grande e organizzato contenitore di dati di alcun tipo*.

1.1 Sistema informativo e sistema informatico

Nelle situazioni sopra elencate, ma anche nella vita di tutti i giorni, è strettamente necessario tenere traccia delle informazioni riguardanti persone o cose. Pensiamo ad esempio ad un'azienda: all'interno di questa sarà necessario tenere traccia di tutti i dettagli anagrafici dei dipendenti, ma anche dei contratti, degli stipendi, ecc...
Tutti questi dati venivano e vengono ancora oggi gestiti dal cosiddetto **sistema informativo**.

Definizione 1.1.1: Sistema informativo

Il *sistema informativo* è l'insieme di quelle risorse, umane o materiali, che un'organizzazione impiega per la gestione e la memorizzazione delle informazioni.
Un banale esempio di sistema informativo potrebbe essere un cassetto contenente delle cartelle, ognuna relativa ad ogni dipendente (risorsa materiale) e la segretaria che è adibita all'accesso e al controllo di questi dati.

Come ben sappiamo però, l'epoca degli archivi fisici è da un po' passata: con l'avvento della tecnologia, e in particolare con la sua massiccia distribuzione, le organizzazioni si sono sempre di più affidate a strumenti di memorizzazione automatizzati e a sistemi per la gestione semi-automatica dei dati (gli elaboratori, per l'appunto). Nasce così un altro importante concetto, quello del **sistema informatico**.

Definizione 1.1.2: Sistema informatico

Il *sistema informatico* è l'insieme delle tecnologie utilizzate dall'organizzazione per la memorizzazione dei dati. Parliamo anche di **sistema informativo automatizzato** quando includiamo nella precedente definizione di sistema informativo anche le componenti elettroniche.

1.2 La gestione dei dati prima dei DBMS

Prima della nascita e della diffusione dei moderni sistemi di gestione di basi di dati (DBMS), i processi di organizzazione e memorizzazione delle informazioni non venivano ancora effettuati nel modo più efficiente possibile. Infatti, ogni organizzazione aveva il proprio **file privato** in cui memorizzare i dati. Utilizzando dunque un singolo file per ogni informazione, la memorizzazione dei dati avveniva in modo *sequenziale*. Dunque, per reperire un dato, era anche necessario scorrere ogni record del file finché questo non fosse stato trovato. Gli svantaggi di questa metodologia sono evidenti ad occhio nudo:

- **Ridondanza:** se lo stesso dato doveva essere condiviso tra più applicazioni, questo doveva essere duplicato (letteralmente un copia e incolla).
- **Inconsistenza:** a causa della ridondanza, aggiornando un'istanza di un record ci si poteva dimenticare di aggiornare anche le altre.
- **Dipendenza dall'applicazione:** ogni applicazione era fortemente legata alla struttura fisica delle cartelle e dei dati in suo possesso.

2

Basi di dati e DBMS

Avendo fornito una panoramica riguardo ai sistemi e alle strategie di memorizzazione utilizzate nel passato (si parla prima degli anni '70), è giusto concentrarsi sullo standard adottato da tali anni fino ad oggi. Iniziamo fornendo una definizione più formale di **base di dati**.

Definizione 2.0.1: Base di dati

Una *base di dati* è una raccolta di dati permanenti, gestita da un elaboratore elettronico.
Più nel pratico, una base di dati rappresenta un insieme di file mutualmente connessi, gestiti efficientemente in modo da rendere l'accesso ai dati il più rapido possibile.
Le basi di dati permettono la memorizzazione di due tipi di dati:

- **Metadati:** delle informazioni speciali che descrivono lo schema della base di dati (la sua struttura) e tutti i vincoli e le relazioni che intercorrono tra i vari file.
- **Dati:** le effettive informazioni che, contestualizzate, vengono inserite e memorizzate nei file. Nota importante sulla parola *contestualizzate*: un numero o una stringa da soli non significano niente. Solo una volta che identifichiamo il numero come ad esempio la matricola di uno studente e la stringa come il suo nome questi dati assumono di significato e possono essere memorizzati con un criterio all'interno dei file.

Assieme al concetto di base di dati presentiamo un altro concetto fondamentale che viaggia di pari passo con il precedente:

Definizione 2.0.2: DataBase Management System (DBMS)

Il *DBMS* è uno strumento software che si occupa principalmente di gestire grandi quantità di dati. Un buon DBMS deve inoltre fornire linguaggi e strumenti per permettere le seguenti operazioni:

- Definizione delle relazioni tra i dati.
- Inserimento/modifica/cancellazione dei dati nei file.
- Creazione dei vincoli di integrità che devono essere stabiliti tra i record in relazione.
- Memorizzazione fisica delle informazioni tramite apposite strutture dati.

Inoltre, un DBMS deve garantire due proprietà fondamentali:

- **Indipendenza fisica:** modificando l'organizzazione della base di dati, i programmi applicativi non devono risentire di modifiche o malfunzionamenti.
- **Indipendenza logica:** i programmi applicativi non devono essere modificati a seguito di una modifica nello schema logico.

Tale principio di indipendenza è fondamentale per eliminare quella dipendenza stretta tra supporto fisico e software che tanto aveva causato problemi nell'epoca pre-DBMS.

Nel mondo dell'informatica, vigono due modalità di uso principali dei DBMS:

On-Line Transaction Processing (OLTP): si tratta della metodologia classica di utilizzo dei DBMS, che prevede lo sfruttamento completo dei vincoli di integrità messi a disposizione da questo. Le organizzazioni che fanno uso di questo paradigma si concentrano maggiormente su *quali dati e come sono memorizzati*, e le operazioni svolte (*transazioni*) sono spesso molto semplici e lavorano su relativamente pochi dati.

On-Line Analytical Processing (OLAP): secondo questo paradigma è importante che i dati siano molti e vengano memorizzati in apposite strutture. Infatti, le organizzazioni che fanno uso di questa modalità tendono a sfruttare le *data warehouses* (letteralmente dei "magazzini per i dati") in quanto sono maggiormente utilizzati nell'ambito della ricerca e dell'analisi scientifica, dove non importa *come* sono organizzati i dati bensì *quali* dati sono memorizzati.

Tuttavia la vera differenza nell'uso dei DBMS venne fatta dal matematica Edgar F. Codd intorno agli anni '70, quando propose di creare un modello di DBMS che si ispirasse al concetto matematico di **relazione** (che ricordiamo essere un sottoinsieme del prodotto cartesiano tra due insiemi). Nasce così il **modello relazionale**.

Definizione 2.0.3: DBMS relazionale

Un DBMS che implementa il modello relazionale organizza i propri dati all'interno di n-uple ordinate, dette **record**, in cui ogni elemento del record rappresenta una particolare informazione.

Nella definizione di una base di dati relazionale bisogna anche definire uno **schema** secondo il quale organizzare questi dati. Tale schema non è altro che la *struttura logica* della tabella, cioè una n-upla di quelli che sono gli attributi (o più informalmente, le colonne) della tabella. Infine, possiamo dire che una tabella contiene tutti record di tipo omogeneo, cioè tutti record che hanno la stessa struttura (o, appunto, lo stesso schema).

All'interno del modello relazionale, due record non vengono collegati tramite puntatori come ad esempio nel modello gerarchico o reticolare (vedi dopo) bensì viene stabilito un collegamento tra due attributi dei due record che devono necessariamente essere dello stesso tipo e contenere lo stesso valore. Intuitivamente si può pensare che tale procedimento causi una forte ridondanza dei dati, ma tale osservazione è parzialmente errata in quanto:

- Un buon DBMS deve mettere a disposizione gli strumenti per la creazione di tali collegamenti e controllare che questi non vengano violati (**vincoli di integrità**).
- Ogni record è memorizzato **una e una sola volta** sul disco fisico. Ciò vuol dire che se un record studente è associato ad una classe, non verrà duplicato alcun record per stabilire il collegamento, ma semplicemente verranno collegati i due record già esistenti. Tale ridondanza deve però essere gestita, come detto anticipatamente, dal DBMS.

Matricola	Cognome	E-Mail
2671072	Rossi	rossi@sapienza.it
2623072	Bianchi	bianchi@sapienza.it
2623272	Verdi	verdi@sapienza.it

Sopra è presentato un esempio di come vengono organizzati i dati all'interno di una tabella nel modello relazionale. Come possiamo vedere ogni record ha la stessa struttura: matricola, cognome, e-mail. Questa struttura inoltre è proprio lo *schema della tabella* (inclusi vari vincoli di integrità stabiliti).

Tuttavia il modello relazionale non fu e ancora oggi non è l'unico modello per la rappresentazione dei dati.

Modello reticolare: i dati vengono presentati come una collezione di record di tipo omogeneo (simile al relazionale). Tuttavia, le effettive *relazioni* tra i vari record venivano fatte tramite dei *puntatori* da un record ad un altro.

Modello gerarchico: si tratta di una sottoclasse del modello reticolare, e prevede l'organizzazione dei dati in una struttura simile a quella di un albero informatico, dove ogni nodo (record) ha solo un genitore.

2.1 Architettura ANSI/SPARC

Nello studio dei sistemi di gestione di basi di dati, risulta particolarmente utile ed efficace distinguere tre differenti livelli di descrizione dei dati, noti comunemente come schemi. Tale suddivisione permette di garantire l'indipendenza dei dati e di gestire la complessità strutturale del sistema. Vediamo più nel dettaglio ciascun livello componente questa architettura.

2.1.1 Schema esterno

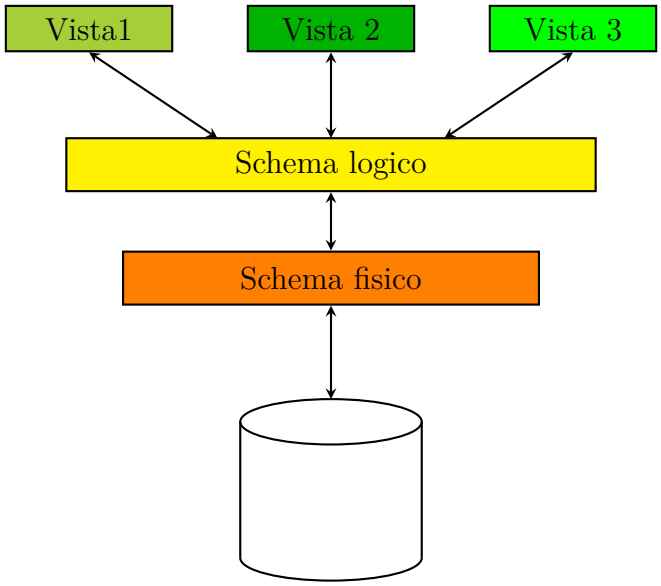
Si tratta della porzione più superficiale dell'architettura, quella con cui saranno a contatto gli utenti che non dovranno avere nulla a che fare con la gestione dei dati, ma solo con il loro utilizzo (ad esempio, la segretaria di un ufficio). Tale schema predispone anche che ogni utente sia in grado di visualizzare solo *una porzione* delle informazioni contenute nella base di dati, in base ai suoi permessi.

2.1.2 Schema logico

Questo livello descrive la struttura degli insiemi di dati e delle relazioni tra loro, secondo il modello adottato dal DBMS. Nota che questo livello si occupa di definire lo schema dei dati, ma non ha alcuna influenza su come questi vengono memorizzati fisicamente in memoria. Un esempio di schema logico può essere il seguente, che definisce la struttura di una tabella all'interno della base di dati di un'università; `Studenti(matricola char(8), cognome char(20), annoDiNascita int)`

2.1.3 Schema fisico

L'ultimo e più profondo livello si occupa di gestire l'effettiva memorizzazione dei dati all'interno dei supporti di memorizzazione, fornendo le strutture dati necessarie per facilitarne l'accesso (in particolare i B-Alberi). Ad esempio, l'amministratore della base di dati potrebbe decidere che i dati vadano *memorizzati* in maniera sequenziale (i record vengono messi uno dopo l'altro) e l'accesso a questi venga fatto tramite un indice sull'identificativo univoco del record.



2.2 Le transazioni

Un buon DBMS deve inoltre gestire anche una serie di operazioni sensibili di inserimento, modifica e cancellazione di dati, in modo che queste vengano effettuate senza contrattempi o errori.

Definizione 2.2.1: Transazione

Una **transazione** è una sequenza di operazioni di lettura e scrittura in memoria che deve rispettare le seguenti proprietà **ACID**:

- **Atomicity:** deve essere una sequenza **indivisibile** di operazioni. In caso di verifica di un qualsiasi errore nel mezzo di una transazione porta questa ad essere marcata come *non valida*, in quanto le operazioni devono essere eseguite **tutte o nessuna**.
- **Consistency:** bisogna lasciare i dati modificati in uno stato corretto che non viola i vincoli imposti precedentemente dal DBMS.
- **Isolation:** ogni transazione deve essere **indipendente**. Dunque durante esecuzione concorrente di più transazioni, gli effetti di una non devono dipendere dalle altre.
- **Durability:** gli effetti di una transazione devono essere **permanenti** per quanto riguarda i dati modificati (ovvero le modifiche devono necessariamente essere applicate anche su disco per mantenere la sincronizzazione dei dati).

Vediamo di seguito un esempio di due transazioni che vengono eseguite bene o male nello stesso momento:

- **Transazione 1**: "Accredita 1000 € sul conto corrente $c1$"
- **Transazione 2**: "Accredita 500 € sul conto corrente $c1$"

Supponiamo che il valore iniziale del saldo del conto $c1$ sia pari a 2000 €. Intuitivamente, al termine di entrambe le transazioni, ci aspetterions un valore finale di 3500 €. Vediamo invece cosa può accadere se le operazioni vengono interlacciate nel tempo senza opportune sue protezioni.

Osservazione 2.2.1

Una volta letto un valore, ogni transazione lo modifica nel **proprio** spazio di memoria di lavoro; altrimenti si rischiano modifiche irreversibili ai dati nello schema.

Analizziamo la seguente tabella di esecuzione temporale delle due transazioni:

Transazione 1	**Tempo**	**Transazione 2**
cerca $c1$	$t1$	
	$t2$	cerca $c1$ ← *qui non prende il valore aggiornato* +1000
modifica *saldo* in *saldo*+1000	$t3$	
	$t4$	modifica *saldo* in *saldo*+500

Come si può vedere dalla tabella, l'esecuzione concorrente di due transazioni, in questo caso, risulta in un esito errato, in quanto ogni transazione effettua modifiche sulla **sua** porzione di dati, e non su quella effettivamente memorizzata, il che causa problemi di sincronizzazione.

Definizione 2.2.2: Serializzabilità

Nel caso di esecuzioni concorrenti di più transazioni, l'effetto complessivo deve essere equivalente a quello di un'esecuzione seriale, ovvero con le transazioni eseguite *una dopo l'altra*, senza alcun interfogliamento delle loro operazioni elementari.

Chiaramente la serializzabilità non è sempre semplice da implementare, e si fa infatti uso di protocolli progettati per garantire a priori la correttezza di qualunque schedule di transazioni che li adottano. Queste tecniche e protocolli non saranno tuttavia argomento di questo corso, per cui si lascia al lettore la voglia di approfondire.

Continuando l'analisi delle funzionalità offerte da un DBMS, è fondamentale esaminare gli strumenti messi a disposizione per la gestione e lo sviluppo. Tali funzionalità non si rivolgono unicamente agli utenti finali che eseguono interrogazioni sui dati, ma coprono ruoli cruciali come l'amministratore della base di dati e lo sviluppatore di applicazioni.

Concludendo questo capitolo, se volessimo elencare i vantaggi e gli svantaggi nell'utilizzo di un DBMS *ben progettato* per la memorizzazione dei dati, otterremmo un simile resoconto.

Osservazione 2.2.2: Vantaggi nell'uso di un DBMS

1. Indipendenza fisica e logica tra dati e applicazioni che li usano.
2. Memorizzazione e recupero efficiente dei dati.
3. Vincoli di integrità sui dati.

Il principale **svantaggio** nell'utilizzo di un DBMS risiede nella stretta necessità di definire uno schema di dati prima di poterli memorizzare. Tale punto diventa uno svantaggio nel momento in cui si vogliono memorizzare, ad esempio, piccole quantità di dati (ad esempio se vogliamo memorizzare il contenuto di un libro), oppure non ci interessa *come* questi dati vengano memorizzati.

3

Progettazione di basi di dati

Progettare una base di dati significa progettare sia la **struttura dei dati** sia le **applicazioni** che vi operano sopra. Tra queste due componenti, la **progettazione dei dati** rappresenta l'attività più importante e delicata. Tale progettazione non è un'attività fine a sé stessa, bensì rientra nel **ciclo di vita di un sistema informativo**.

Definizione 3.0.1: Ciclo di vita di un sistema informativo

Il *ciclo di vita di un sistema informativo* è quella sequenza di attività svolte da analisti, progettisti e utenti nello sviluppo dei sistemi informativi. Viene detto *ciclo* in quanto ogni fase non è una tantum, bensì può essere ripetuta più volte apportando le opportune modifiche al fine di raggiungere il risultato finale sperato.

Tale ciclo si articola nelle seguenti fasi:

1. **Studio di fattibilità:** definizione delle proprietà, stime dei costi, valutazione dei tempi richiesti dal progetto.
2. **Raccolta e analisi dei requisiti**
3. **Progettazione e implementazione:** modellazione concettuale e logica dei dati che si vogliono memorizzare e implementazione tramite un apposito linguaggio DBMS.
4. **Validazione e collaudo:** effettuazione dei test per verificare che il sistema risponda correttamente alle esigenze.

La modellazione di una base di dati si colloca a *metà* tra la raccolta e l'analisi dei requisiti e la progettazione effettiva.

3.1 Modelli informatici

Prima di modellare la base di dati, bisogna decidere a quale modello informatico/matematico rifarsi per la memorizzazione di questi. Un modello deve presentare le seguenti caratteristiche:

- Deve essere **la rappresentazione di un certo fatto concreto**.
- Tale rappresentazione deve essere descritta mediante un **linguaggio formale** (dunque che segue delle regole ben definite).
- Individuazione delle strutture da utilizzare per organizzare i dati di interesse e le relazioni tra questi.
  Ad esempio, il modello relazionale di cui abbiamo discusso prima utilizza il concetto matematico di *relazione* per organizzare i dati.

Possiamo definire due tipologie di modelli fondamentali nella progettazione di una base di dati:

Definizione 3.1.1: Modello logico e modello concettuale

Il **modello logico** di una base di dati rappresenta la struttura dei dati e come questi vengono memorizzati. Si potrebbe dire che corrisponde allo schema logico visto precedentemente nel modello ANSI/SPARC, in quanto si colloca indipendente dalla memorizzazione fisica dei dati, ma è strettamente legato dal tipo di modello informatico che abbiamo scelto di adottare per la memorizzazione di questi.

Il **modello concettuale** è tutto il contrario: deve essere completamente indipendente dalle modalità di realizzazione, e ha lo scopo di rappresentare la realtà descritta dal problema, tramite un preciso linguaggio formale (ad esempio i diagrammi UML, ma anche il modello **Entity-Relationship (E-R)**)

Non esiste un modello corretto: la stessa realtà può essere modellata in modi diversi a seconda dell'interpretazione che si dà al problema. Tuttavia deve essere seguito uno specifico "percorso mentale" nella progettazione di una base di dati:

1. **Aspetto ontologico:** quanto conosco del dominio del discorso? Quanto sono familiare con l'ambito di discussione?
   Durante questa fase, detta anche fase di *analisi del problema*, devo delineare quali sono le entità la cui realtà deve essere modellata e come si relazionano tra loro.

2. **Aspetto logico:** con quale concetto astratto voglio realizzare le entità del modello? (es. relazionale, gerarchico, reticolare, ...)
   Durante questa fase definisco le effettive relazioni tra le entità.

3. **Aspetto linguistico:** con quale linguaggio formale definiamo il modello? (es. SQL/algebra relazionale)

4. **Aspetto pragmatico:** come memorizzo i dati nei supporti di memorizzazione fisica?

3.2 Entità, collezioni e attributi

Nella descrizione delle fasi da seguire durante la progettazione di una base di dati abbiamo citato un termine importante e fondamentale nella modellazione concettuale della realtà di riferimento: **entità**.

Definizione 3.2.1: Entità

Le *entità* di un problema sono quegli oggetti a cui il problema dà particolare rilevanza, e dei quali ci interessa rappresentare alcuni fatti a loro associati, detti per l'appunto **attributi (o proprietà) dell'entità**.
Ogni entità appartiene ad un **tipo** che ne specifica la natura. Ad esempio l'entità `Studente` appartiene alla categoria `Persona` (possiamo vederla come l'ereditarietà nella programmazione ad oggetti).
Una **collezione** di entità è invece un insieme variabile nel tempo di entità tutte dello stesso tipo. Ad esempio, se pensiamo a tutti gli studenti di un'università, quella rappresenta una collezione di entità di tipo `Studente`.

Supponiamo di dover creare la base di dati di un sistema di memorizzazione degli studenti, professori ed esami. Possiamo individuare nello studente un'**entità**, in quanto si tratta di uno degli oggetti fondamentali descritti dal problema, e nella sua matricola, cognome e data di nascita gli **attributi** fondamentali che lo descrivono.

Un entità o un attributo non mantengono il loro status costantemente all'interno di ogni analisi e di ogni problema: in una realtà potrebbe essere messa più importanza su un oggetto, spingendoci dunque a farlo diventare entità, mentre in un'altra tale importanza potrebbe mancare, rilegando così l'oggetto ad un semplice attributo. Ad esempio, proviamo a rappresentare una biblioteca: se ci interessasse modellare solo i libri, senza curarci degli autori, potremmo avere uno schema logico di questo tipo `Libro(titolo, annoDiPubblicazione, casaEditrice, autore)`. Se invece fossimo interessati anche agli

autori, renderemmo quell'oggetto una vera e propria entità con nome e cognome, anziché lasciarlo un semplice attributo.

3.3 Le associazioni

Definizione 3.3.1: Associazione

Un'**istanza di associazione** è un fatto che collega due o più entità, stabilendo un legame logico fra loro.
Matematicamente, definiamo $R(X, Y)$ un'associazione fra due collezioni di entità X ed Y un insieme di istanze di associazione tra elementi $x \in X$ e $y \in Y$. Il prodotto cartesiamo $X \times Y$ è detto **dominio dell'associazione**.

Generalmente, nella modellazione concettuale, le associazioni sono descritte da un *verbo* che lega le varie entità coinvolte. Chiaramente tale verbo deve essere esplicativo della natura dell'associazione, e per tale ragione si devono evitare verbi come "essere" o "avere" oppure verbi modali.
Un esempio di associazione possiamo vederlo anche con il precedente esempio della biblioteca: le entità `Libro` e `Autore` sono collegate tra loro dall'associazione `SCRIVERE`, in quanto è l'autore a scrivere il libro, e il libro viene scritto dall'autore.

Un'associazione è caratterizzata dalle seguenti proprietà strutturali:

- **Molteplicità:** prese le due collezioni di entità X ed Y e un'associazione $R(X, Y)$, indica quante istanze di un elemento $x \in X$ sono associate ad uno e un solo elemento $y \in Y$.
  Formalmente, tale proprietà è descritta dal **vincolo di univocità**:
  - Un'associazione si dice **univoca** rispetto ad X se per ogni elemento $x \in X$ esiste **al più** un elemento $y \in Y$ che è associato ad x.
  - Se tale vincolo fallisce (ovvero se esiste **almeno** un elemento $x \in X$ a cui è associato più di un elemento $y \in Y$) l'associazione si dice **multivalore**.

  Facendo un esempio, un'associazione del tipo $R(X, Y) = (1 : N)$ è multivalore su X e univoca su Y. Viceversa, se avessimo scritto $R(X, Y) = (N : 1)$, sarebbe stata univoca su X e multivalore su Y. Possiamo leggerla "a croce", diciamo: la molteplicità dell'elemento più a sinistra è il valore più a destra, mentre quella dell'elemento più a destra è il valore più a sinistra. Come nelle proporzioni, gli esterni vanno con gli esterni e gli interni con gli interni.

- **Totalità:** indica se l'associazione tra un elemento $x \in X$ e un elemento $y \in Y$ è opzionale oppure no. Formalmente:
  - Un'associazione $R(X, Y)$ si dice **totale** se, per ogni elemento $x \in X$ esiste *almeno* un elemento $y \in Y$ associato ad x (questa proprietà è conosciuta meglio come **obbligatorietà del valore**).
  - Tale associazione si dice invece **parziale** quando il vincolo imposto precedentemente fallisce, ovvero quando ad un valore $x \in X$ *non è detto* che vi sia associato un valore $y \in Y$ (questa proprietà è più conosciuta come **opzionalità**).

  Ad esempio, pensiamo ad un'associazione del tipo `Possedere(Persona, AutomobileConTarga)`. Riprendendo il discorso sulla molteplicità, questa sarà del tipo `(1:N)`, in quanto una persona può possedere più di un'automobile con targa, però una singola automobile con targa è posseduta da una sola persona. Vedendo invece il vincolo di totalità, quest'associazione risulta essere *opzionale* sulla Persona, in quanto non è detto che una persona possegga un'automobile, mentre è *totale* su automobile, in quanto un'automobile già targata è sicuramente proprietà di qualcuno.

Vediamo adesso una delle numerose rappresentazioni grafiche con cui possiamo rappresentare le associazioni, e delle particolarità che le contraddistinguono:

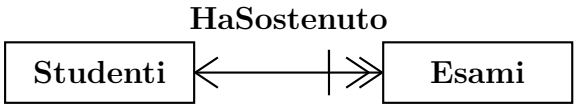


Figura 3.1: Relazione **HaSostenuto**

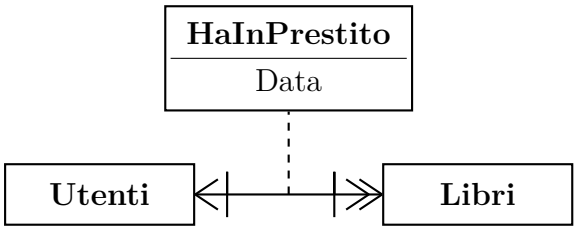


Figura 3.2: Relazione con attributi. L'attributo *Data* è agganciato alla relazione stessa.

Descriviamo adesso tutte le informazioni ricavabili dalla notazione presa in esempio:

- Il singolo `>` oppure `<` significa che la relazione è **univoca** verso quel lato. Vedendo la figura 3.1 vediamo come in questa specifica modellazione un esame venga sostenuto da uno e un singolo studente.

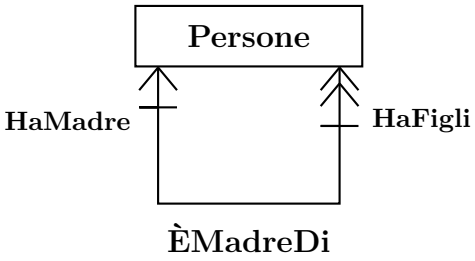


Figura 3.3: Relazione ricorsiva. L'entità **Persone** si relaziona con sé stessa tramite i ruoli *HaMadre* e *HaFigli*.

- Il doppio » oppure « significa che da quel lato la relazione è **multivalore**. Sempre prendendo in esempio gli studenti, uno studente può aver sostenuto più volte lo stesso esame. Anche con `Utenti - Libri`, vediamo come un utente prende in presito più libri, mentre un libro viene preso in prestito solo da un utente alla volta.
- La barra verticale `|` vicino alla molteplicità indica che da quel lato la relazione è **parziale**, dunque opzionale. Ad esempio vediamo come, nel caso `Studenti - Esami` il singolo studente potrebbe ancora non aver mai sostenuto quell'esame, mentre l'esame stesso, essendo registrato, è sicuramente stato sostenuto da qualche studente.

3.4 Le gerarchie di classi

Spesso le classi di entità sono organizzate in una gerarchia di *specializzazione/generalizzazione*. Una classe della gerarchia minore di altre viene detta **sottoclasse**, mentre le altre prendono il nome di **superclassi**. Il concetto è lo stesso alla base dell'ereditarietà nella programmazione ad oggetti. Per fare un esempio pertinente, la classe degli *studenti universitari* può è una generalizzazione (o sopraclasse) delle seguenti classi:

- Matricole e laureandi.
- Studenti in corso e studenti fuori corso.
- Studenti locali e studenti fuori sede.
- Studenti maschi e studentesse.

Fra i tipi oggetto è definita una **relazione di sottotipo** che regola la struttura gerarchica dei dati. Per il lettore questa è semplicemente una formalizzazione matematica del concetto di *polimorfismo* all'interno della programmazione ad oggetti.

Definizione 3.4.1: Relazione di sottotipo

Fra i tipi oggetto viene stabilita una relazione di sottotipo che soddisfa le seguenti proprietà fondamentali:

- È **asimmetrica**, **riflessiva** e **transitiva** (definendo così una relazione di ordine parziale).
- **Principio di sostitutività**: Se T è sottotipo di T', allora gli elementi appartenenti al tipo T possono essere usati in qualsiasi contesto in cui possano apparire valori di tipo T'.

  In particolare, ciò implica che:

  – Gli elementi di T possiedono tutte le proprietà degli elementi di T'.

  – Per ogni proprietà p in T', il suo tipo associato in T deve essere un sottotipo del suo tipo in T'.

Facendo nuovamente riferimento alla programmazione ad oggetti, anche nella modellazione concettuale possiamo definire degli oggetti (le entità del problema che vogliamo modellare) a partire da altri già esistenti. Questi nuovi oggetti erediteranno dal proprio genitore (o *dai propri genitori*, in quanto viene supportata l'ereditarietà multipla) tutti i suoi attributi, e potranno anche definirne di nuovi o ridefinire gli esistenti.

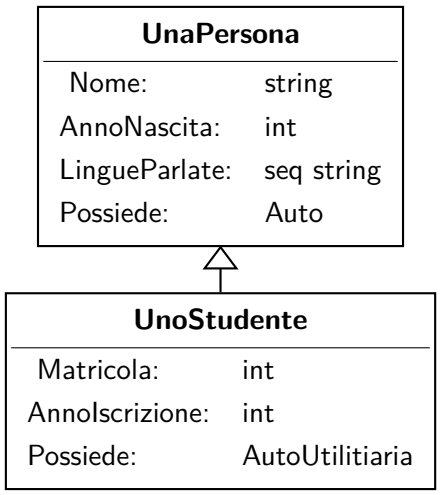


Figura 3.4: Ereditarietà tra classi. La classe **UnoStudente** eredita da **UnaPersona**, specializzando il tipo dell'attributo *Possiede*.

Fra le classi può essere definita una relazione di **sottoclasse**, detta anche di **sottoinsieme**, caratterizzata dalle seguenti proprietà:

Definizione 3.4.2: Relazione di sottoclasse

La **relazione di sottoclasse** è definita dalle seguenti proprietà:

- È **antisimmetrica**, **riflessiva** e **transitiva**.
- Se C è sottoclasse di C', allora il tipo degli elementi di C è sottotipo del tipo

degli elementi di C' (**vincolo intensionale**).

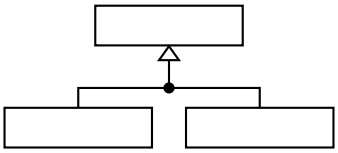
- Se C è sottoclasse di C', allora gli elementi di C sono un sottoinsieme degli elementi di C' (**vincolo estensionale**).

3.4.1 Vincoli su insiemi di sottoclassi

Per gli insiemi delle sottoclassi (i domini dei valori che tali classi possono assumere) vengono anche stabiliti dei vincoli.

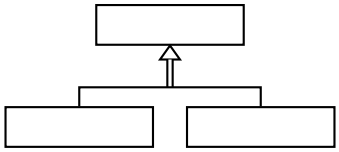
Definizione 3.4.3: Vincolo di disgiunzione

Gli elementi di una sottoclasse non possono far parte di un'altra sottoclasse.
Sostanzialmente stiamo dicendo che, avendo delle classi che sono specializzazioni di una sopraclasse, tali sottoclassi **non avranno nulla in comune**, e dunque un elemento di una sottoclasse non avrà nulla a che vedere con un elemento di un'altra sottoclasse.
Pensiamo ad esempio di avere una classe `Persona` che ha come sottoclassi `Maggiorenni` e `Minorenni`. Tra queste sottoclassi vige il vincolo di disgiunzione, in quanto una persona non potrà **mai** essere contemporaneamente sia minorenne che maggiorenne. D'altro canto, se avessimo una classe `Persona` con sottoclassi `Programmatore` e `Maggiorenne`, tra queste due sottoclassi non vige il vincolo di disgiunzione in quanto non possiamo categoricamente dire che nessun programmatore è maggiorenne o viceversa. Formalizzando in maniera matematica questo vincolo, possiamo dire che *i domini delle due sottoclassi sono due insiemi la cui intersezione è nulla.*



Definizione 3.4.4: Vincolo di copertura

L'unione degli elementi delle sottoclassi contiene **tutti** gli elementi della sopraclasse.
Mantenendoci sempre sull'esempio fatto in precedenza con `Persona, Maggiorenne, Minorenne`, tra le due sottoclassi è presente un vincolo di copertura, in quanto una persona **o** è maggiorenne **o** è minorenne. Necessariamente una delle due. In modo più matematico e formale, possiamo dire che *l'unione dei domini delle sottoclassi è uguale al dominio della sopraclasse.*



Se tra le sottoclassi di una gerarchia si verificano contemporaneamente il vincolo di disgiunzione e quello di copertura, si dice che l'insieme delle sottoclassi è una **partizione** della classe padre.

3.5 Le fasi per la costruzione di una base di dati

Il ciclo di vita e il processo di sviluppo di una base di dati si articola nelle seguenti fasi principali:

- **Analisi dei requisiti**: consiste nell'analizzare il problema e comprenderne a fondo le specifiche fornite dall'utente o committente, tipicamente in linguaggio naturale.
- **Progettazione**: si suddivide a sua volta nella progettazione dei dati e nella progettazione delle applicazioni.
  - **Progettazione concettuale**: ha lo scopo di individuare le entità di interesse e le associazioni tra di esse, formalizzandole tramite un apposito modello concettuale ad alto livello (ad esempio il modello E-R o i diagrammi delle classi UML) che serve a descrivere la realtà di riferimento da un punto di vista abbastanza astratto, in maniera totalmente indipendente da quella con cui saranno organizzati i dati dal DBMS.
  - **Progettazione logica**: consiste nella traduzione dello schema concettuale nel modello logico scelto (ad esempio il modello relazionale), definendo puntualmente le strutture dati logiche e le relazioni intercorrenti tra esse.
  - **Progettazione fisica dei dati**: riguarda gli aspetti implementativi di basso livello, definendo l'organizzazione fisica e le strutture di memorizzazione dei dati su disco (file di memorizzazione, indici, metodi di accesso).
  - **Progettazione delle applicazioni**: definizione dell'architettura e della logica dei programmi software che interagiranno con la base di dati.
- **Realizzazione**: fase in cui il database viene effettivamente implementato sul DBMS prescelto, popolato e integrato con le applicazioni software.

4

Progettazione concettuale

Nel corso del seguente capitolo analizzeremo la prima fase della progettazione di una base di dati, forse la più fondamentale, che ci permetterà di tracciare le basi per la creazione delle relazioni e delle tabelle in memoria.

Durante la fase di progettazione concettuale, in cui si va a delineare il **modello concettuale** derivato dall'analisi del problema, è prevalente l'**aspetto intensionale della progettazione**.

Definizione 4.0.1: Aspetto intensionale

L'aspetto intensionale è quel lato della progettazione che definisce quali entità esistono, quali attributi hanno e quali vincoli devono rispettare. E' **invariante nel tempo**.

Il modello concettuale più diffuso è sicuramente il **modello E-R**, del quale parleremo maggiormente nella prossima sezione.

4.1 Il modello E-R

Il modello E-R è un modello grafico utilizzato per descrivere la struttura astratta di una base di dati, indicando gli oggetti coinvolti e le relazioni tra questi. Mette le basi su alcuni costrutti fondamentali.

4.1.1 Entità

Definizione 4.1.1: Entità

Classi di oggetti omogenei tra loro e con esistenza autonoma. Ciò vuol dire che un'entità, per esistere, non deve essere necessariamente collegata o dipendente da qualcos'altro. Sono, insieme alle associazioni, l'aspetto più importante del problema, dato che, informalmente parlando, le entità si trasformeranno nelle *tabelle* durante la progettazione del modello logico. Un esempio di entità potrebbe essere un `Professore` o un `Corso` nell'ottica di una base di dati universitaria. L'entità è quell'oggetto a cui, nella descrizione della realtà di riferimento, si attribuisce particolare importanza e a cui è richiesto che vengano fatte afferire delle informazioni (gli attributi).

Ogni entità ha un **nome identificativo** (che può essere qualsiasi, ma ovviamente si tende a scegliere nomi che facciano capire cosa si sta modellando) che per convenzione si scrive al **singolare**.

E' importante distinguere il concetto di entità da quello di **istanza**, in quanto quest'ultima non rappresenta una classe di oggetti dello stesso tipo bensì **il singolo oggetto, il singolo elemento della classe**. Nel modello concettuale rappresentiamo le entità, non le singole istanze.

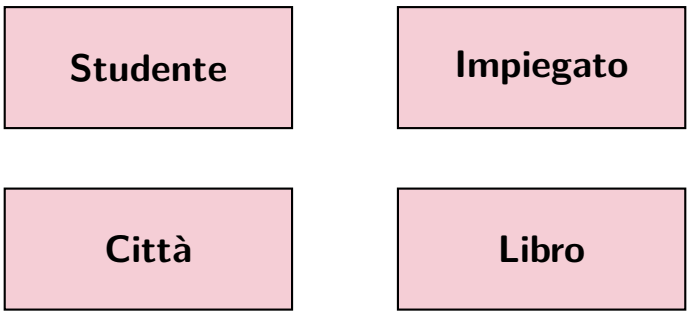


Figura 4.1: Un esempio della rappresentazione grafica delle entità nel modello E-R

4.1.2 Associazioni

Definizione 4.1.2: Associazione

Legame logico che si stabilisce tra due o più entità che hanno qualcosa in comune. Queste *non* sono autonome, in quanto affinché esista un'associazione è necessario ci siano almeno 2 entità coinvolte in questa. Mantenendo l'esempio di `Professore` e `Corso`, un'associazione che li lega potrebbe essere `Insegnamento`.

A seconda della convenzione utilizzata, le associazioni potrebbero essere indicate da un sostantivo oppure da un verbo.

Anche qui, è importante distinguere tra associazione e **occorrenza**, in quanto quest'ultima individua il singolo elemento della classe di legami, identificato dalle istanze delle entità che collega. In generale un'associazione si dice **n-aria** quando collega contemporaneamente n entità.

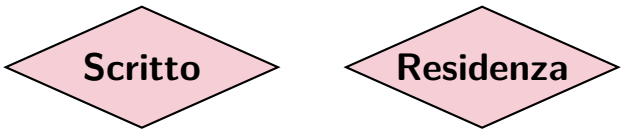


Figura 4.2: Rappresentazione grafica di associazioni senza le entità coinvolte.

Vediamo adesso alcune categorie particolari di associazioni che si potrebbero incontrare nella scrittura del modello E-R.

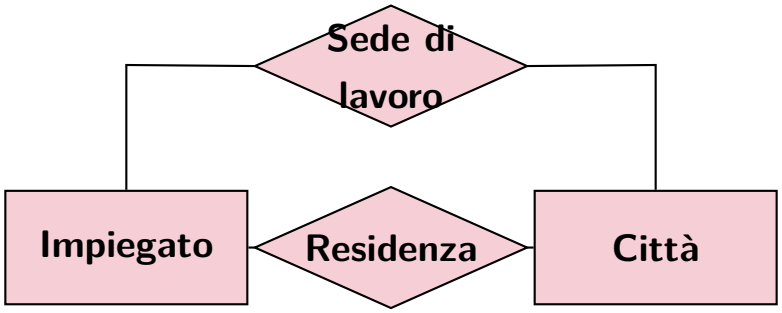


Figura 4.3: Multiple associazioni sulle stesse entità

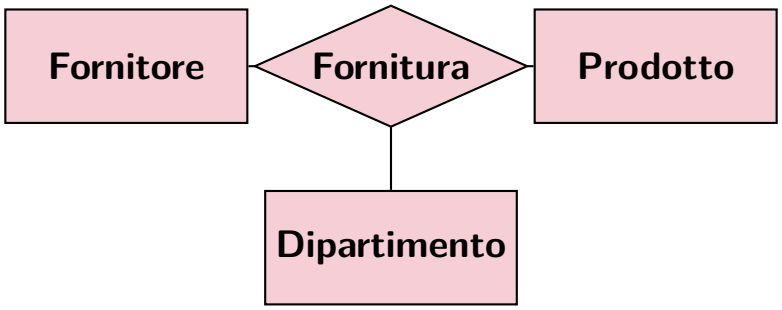


Figura 4.4: Associazione n-aria (in questo caso ternaria).

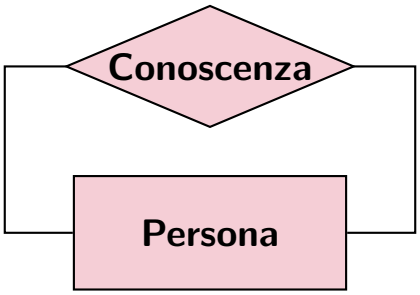


Figura 4.5: Associazione ricorsiva, cioè che coinvolge un'entità con sé stessa.

4.1.3 Attributi

Definizione 4.1.3: Attributo

Proprietà elementare, caratteristica di un'entità o di un'associazione. Si potrebbe pensare agli attributi del modello concettuale in modo similare agli attributi di classe nella programmazione ad oggetti.

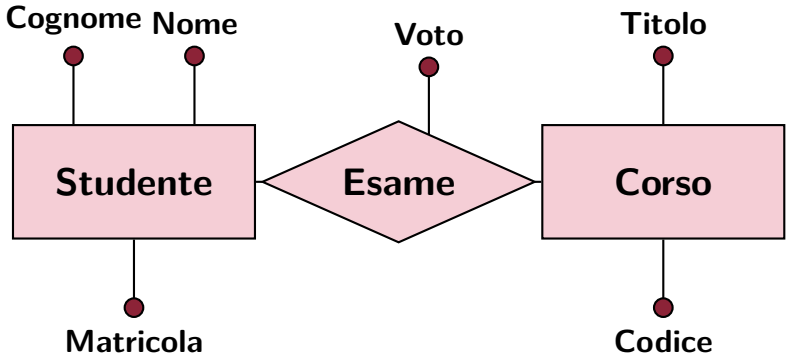


Figura 4.6: Modello E-R con entità, associazioni, e i loro attributi.

Gli attributi possono anche essere **composti**, cioè attributi che raggruppano più informazioni insieme. Un esempio di attributo composto potrebbe essere l'indirizzo di residenza di una persona, in quanto l'informazione indirizzo può essere ulteriormente scomposta in *Via*, *Numero civico* e *CAP*, ad esempio. In un'applicazione la scelta di mantenere un attributo composto anziché suddividerlo nelle sue componenti atomiche dipende dall'importanza che quell'attributo in seguito all'analisi del problema.

4.1.4 Cardinalità di associazioni

Definizione 4.1.4: Cardinalità di associazioni

Si tratta di una coppia di valori che si attribuisce ad ogni entità coinvolta in un'associazione. Si tratta di una rappresentazione grafica dei concetti di **univoco** e **multivalore** visti precedentemente.

Specificano il numero minimo e massimo di occorrenze dell'associazione a cui un'istanza della prima entità può essere collegato rispetto ad istanze della seconda entità.

Vediamo adesso più nel dettaglio le varie tipologie di cardinalità:

- **Partecipazione minima**
  - **0**: indica che la partecipazione di un'istanza dell'entità di partenza all'associazione è **opzionale**. Ciò vuol dire che potrebbe esserci anche una singola occorrenza dell'entità di partenza che non è associata a nessuna occorrenza dell'entità di arrivo. E' il corrispettivo dell'associazione **parziale** che avevamo già visto.
  - **1**: indica che ogni istanza dell'entità di partenza è associata ad almeno un'occorrenza dell'altra entità. E' il corrispettivo dell'associazione **totale** che avevamo già visto.
- **Partecipazione massima**:
  - **1**: indica che ogni occorrenza dell'entità di partenza è collegata al massimo ad un'occorrenza dell'entità di arrivo. E' il corrispettivo dell'associazione **univoca** che avevamo già visto.
  - **N**: indica che ogni occorrenza dell'entitò di partenza può essere collegata a più occorrenze dell'entità di arrivo. E' il corrispettivo dell'associazione **multivalore** che avevamo già visto.

Con riferimento alle cardinalità appena mostrate, possiamo distinguere varie tipologie di associazioni:

- **Uno ad uno**, si verifica quando la cardinalità dell'entità è del tipo (1, 1).
- **Uno a molti**, si verifica quando la cardinalità è del tipo (1, N).
- **Molti a molti**, si verifica quando entrambe le entità coinvolte hanno cardinalità (0, N) oppure (1, N).



Figura 4.7: Relazione tra un tutor e un docente, che si legge nel seguente modo: Un tutor fa il tutoraggio ad 1 massimo 2 docenti. Un docente può avere da 0 a 5 tutor che fanno tutoriaggio.

E' possibile delineare la cardinalità anche per gli attributi: quando questa non viene indicata è implicitamente (1, 1).

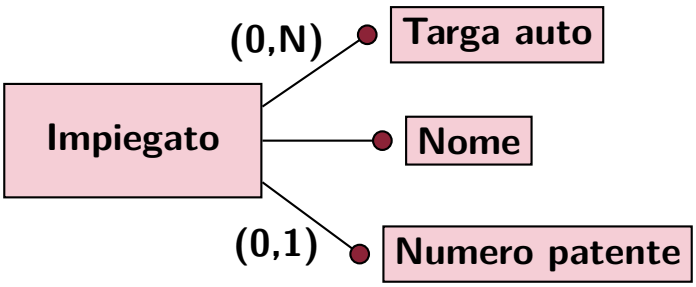


Figura 4.8: Esempio di cardinalità su attributi. Come per le associazioni, scrivere 0 come cardinalità minima indica che l'attributo non è obbligatorio.

4.1.5 Identificatore di un'entità

Definizione 4.1.5: Identificatore

L'identificatore è quell'attributo o quell'insieme di attributi che permettono di identificare univocamente un'istanza di entità.
Questi identificatori possono essere costituiti da:

- Uno o più attributi interni all'entità.
- Uno o più attributi esterni ottenuti attraverso associazioni.

Osservazione 4.1.1: Natura degli identificatori

- Un entità deve avere **almeno** un identificatore, ma può averne anche più di uno.
- Un identificatore esterno è possibile solo per entità che partecipano ad un'associazione con cardinalità (1, 1).

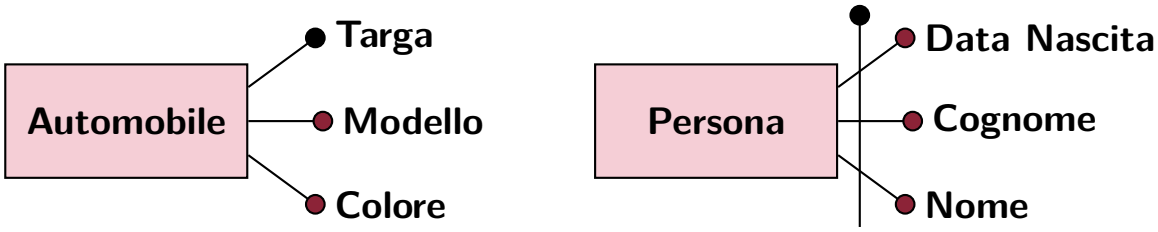


Figura 4.9: Esempio di identificatori interni semplici e composti.

- Per le associazioni non occorre un identificatore in quanto non rappresentano un fatto bensì un legame logico.

4.1.6 Generalizzazioni

Definizione 4.1.6: Generalizzazione

Una generalizzazione è un costrutto che mette in relazione una o più entità $E_1, E_2, \ldots, E_n$ con un'entità E che le comprende come casi particolari. In poche parole, le entità $E_1, \ldots, E_n$ sono delle sottoclassi (o **specializzazioni**) dell'entità E, e dunque ereditano tutte le caratteristiche dell'entità padre potendo poi definirne di nuove.

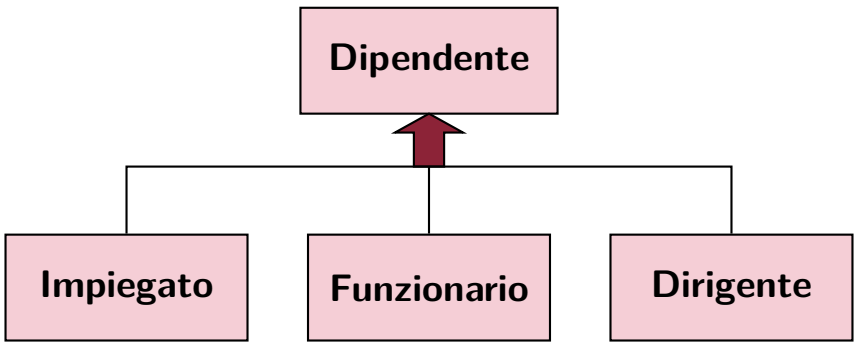


Figura 4.10: Gerarchia di generalizzazione.

Una generalizzazione può essere

Totale: quando ogni occorrenza dell'entità padre appartiene ad almeno un'occorrenza di una delle specializzazioni (in parole povere, quando un istanza del padre è del tipo di almeno un figlio). Un esempio potrebbe essere, `Persona <- Maggiorenne, Minorenne`. In questo caso è totale in quanto una persona è maggiorenne oppure minorenne.

Parziale: quando esiste almeno un'occorrenza dell'entità padre che non appartiene ad alcuna categoria. Ad esempio `Animale <- Cane, Gatto`. Ovviamente cani e gatti non sono gli unici animali esistenti.

Esclusiva: quando ogni occorrenza del padre può appartenere al massimo ad una categoria. Vale l'esempio di prima dei maggiorenni e minorenni.

Sovrapposta: quando esiste almeno un'istanza dell'entità padre che appartiene a più categorie contemporaneamente. Un esempio in questo caso potrebbe essere `Persona <- Studente, Lavoratore`, in quanto una persona può contemporaneamente essere uno studente ma anche un lavoratore.

Concluso il discorso sui costrutti che riguardano il modello E-R, spendiamo due parole anche per parlare della documentazione da associare quando si sviluppa un modello concettuale.

Dizionario delle entità: consiste in una tabella in cui si riportano le entità descritte nel modello assieme ad alcune informazioni importanti riguardanti queste: **nome, breve descrizione, attributi, identificatore univoco**.

Dizionario delle associazioni: come quello delle entità, si tratta di una tabella volta a descrivere tutte le associazioni presenti nel modello. Si includono informazioni come: **nome, breve descrizione, entità coinvolte, attributi dell'associazione**.

Vincoli non esprimibili: il modello concettuale è un ottimo strumento per rappresentare la realtà di riferimento del problema, tuttavia non è abbastanza esaustivo per quanto riguarda eventuali vincoli stabiliti sulla base di dati. Nell'ottica di una base di dati aziendale si potrebbe, ad esempio, impedire che un dirigente abbia uno stipendio inferiore a quello di un impiegato normale (ci rendiamo conto che è impossibile esprimere tale vincolo sul modello E-R).

5

Progettazione logica

Come seconda fase nella progettazione di una base di dati abbiamo la progettazione logica, in cui ci avviciniamo maggiormente all'architettura del DBMS e per questo bisogna definire il modello con cui si voglio opportunamente memorizzare i dati.
In questo corso faremo esclusivamente uso del modello relazionale, che a breve vedremo con maggior dettaglio.

5.1 Il modello relazionale

Il modello relazionale si basa sul concetto matematico di **relazione**: tali relazioni si traducono in maniera naturale in tabelle (infatti talvolta i due termini potrebbero essere scambiati). Prima di passare alla definizione formale di questo modello, vediamo alcune definizioni di oggetti matematici che ci serviranno:

Definizione 5.1.1: Dominio

Un **dominio** è un insieme potenzialmente infinito di valori. Ricordando lo studio di una funzione, il dominio di questa era l'insieme dei valori $x \in \mathbb{R}$ a cui la funzione associava un valore.

Possiamo scegliere anche esempi più comuni di dominio: l'insieme dei numeri naturali, l'insieme dei valori in un sistema di numerazione binario (banalmente solo 0 e 1), l'insieme delle stringhe lunghe 20 caratteri, ...

Definizione 5.1.2: Prodotto cartesiano

Siano $D_1, D_2, \ldots, D_k$ domini non necessariamente distinti. Il **prodotto cartesiano** tra tali domini è denotato come

$$D_1 \times D_2 \times \cdots \times D_k$$

che, insiemisticamente, può essere visto come quella sequenza ordinata di valori in cui ogni valore è preso da un dominio diverso.

$$\{(v_1, v_2, \ldots, v_k) | v_1 \in D_1, v_2 \in D_2, \ldots, v_k \in D_k\}$$

Definizione 5.1.3: Relazione matematica

Una **relazione**, in matematica, è un qualsiasi **sottoinsieme del prodotto cartesiano di uno o più domini**.

Il **grado di una relazione** è il numero di domini coinvolti nel prodotto cartesiano che hanno prodotto quella relazione (dunque se abbiamo il prodotto cartesiano tra 5 domini e prendiamo una relazione su quel prodotto, il suo grado sarà 5).

Gli elementi di una relazione sono detti **tuple** o **n-uple**, e il numero di n-uple di una relazione rappresenta la sua **cardinalità**.
L'ordine fra gli elementi di una tupla (che sono presi ognuno da ogni dominio, in modo **ordinato**) è **importante**, mentre è irrilevante l'ordine delle tuple di una relazione (essendo essa un insieme).

Supponiamo di avere due domini: $D_1 = \{\text{bianco, nero}\}, D_2 = \{1, 2, 3\}$.
Il prodotto cartesiano sarà

$$D_1 \times D_2 = \{\text{(bianco, 1), (bianco, 2), (bianco, 3), (nero, 1), (nero, 2), (nero, 3)}\}$$

Presa una relazione $R = \{\text{(bianco, 1), (nero, 1), (nero, 3)}\}$, questa avrà grado 2 (in quanto è stata generata dal prodotto cartesiano di 2 domini) e cardinalità 3 (in quanto ha 3 elementi).

Forniamo adesso dell'altra notazione riguardante relazioni e tuple. Sia R una relazione di grado k. Sia $t \in R$. Sia $i \in \{1, \ldots, k\}$. Denotiamo con $t[i]$ oppure $t.i$ l'**i-esima componente di t**. Per l'esempio di prima, supponiamo $t = \{(\text{nero}, 1)\}$. Abbiamo che t[1] = nero, t[2] = 1, t[1,2] = (nero, 1).

Abbiamo già detto che nei database relazionali c'è una fortissima corrispondenza tra il concetto di tabella e quello di relazione. Il problema è che se ci viene data una tabella così senza descrizioni, decifrarne il contenuto diventa quasi impossibile.

E' per questo che nasce l'importanza di definire degli attributi nelle tabelle.

Definizione 5.1.4: Attributo

Un **attributo** è definito da un **nome** `A` e dal **dominio** di tutti i valori che `A` può assumere.

Definizione 5.1.5: Schema di relazione e tuple

- Sia `R` un insieme di attributi diversi (**schema di relazione**, sarebbe l'intestazione della tabella). Una n-upla su `R` è una **funzione che associa ad ogni attributo A in R un elemento del dominio di A**.
  In poche parole, una n-upla, quella che informalmente chiameremmo *riga della tabella* o *istanza della tabella* viene matematicamente definita come una funzione che ad ogni colonna associa il suo effettivo valore, preso appunto dal dominio di valori consentiti dall'attributo.
- Se `t` è una n-upla su `R` ed `A` è un attributo di `R`, indichiamo con `t(A)` il valore assunto dalla funzione `t` in corrispondenza dell'attributo `A`.
  Sempre in parole molto povere, facendo `t(A)` ci viene restituito il valore che l'attributo `A` assume nella riga `t`.
- Se $Y \subseteq X$, dove X è lo schema di una relazione R, denotiamo con t[Y] l'insieme dei valori assunti dagli attributi $y \in Y$ nella tupla t.

Infine, possiamo definire lo **schema di una base di dati relazionale** come l'insieme di tutti gli schemi di relazione che hanno nomi differenti.

Città	**Regione**	**Popolazione**
Roma	Lazio	3000000
Milano	Lombardia	1500000
Genova	Liguria	800000
Pisa	Toscana	150000

Tabella 5.2: Dati relativi a città, regione di appartenenza e popolazione

5.1.1 Chiavi di una relazione

Abbiamo già implicitamente parlato di chiavi quando, nel modello concettuale, citavamo gli identificatori di un'entità. Avendo però ora definito più concetti, possiamo fornire una definizione leggermente più formale di chiave.

Definizione 5.1.6: Chiave

Una **chiave** è un attributo o un insieme di attributi che identifica univocamente una tupla.

Supponendo che `X` sia un insieme di attributi che formano una chiave, valgono le seguenti proprietà:

- Se $t_1, t_2, t_1 \neq t_2$ tuple di una relazione R, allora $t_1[X] \neq t_2[X]$.
- X è il più piccolo insieme che soddisfa la proprietà precedente.

La chiave composta dal minor numero di attributi possibili ad identificare una tupla si dice **chiave primaria**. Questa gode di due fondamentali proprietà:

- Non può avere valori nulli.
- Consente di mettere in relazione tabelle diverse.

5.2 Ristrutturazione del modello concettuale

Come mai la fase di ristrutturazione del modello concettuale appartiene alla progettazione logica?
Ricordando quando detto nei primi capitoli, è molto importante progettare una base di dati mantenendo i paradigmi di indipendenza fisica e logica. Se nella prima fase di progettazione concettuale abbiamo descritto la realtà in modo completamente astratto e indipendente dalla memorizzazione fisica dei dati, con questa fase di ristrutturazione dovremo andare ad analizzare il modello concettuale già nell'ottica di voler organizzare i dati in tabelle e relazioni tra queste. Questo avviene in quanto l'obiettivo principale della progettazione logica è quello di **tradurre il modello concettuale in uno schema logico che rappresenti i dati nella stessa maniera**. Questa fase di ristrutturazione si articola in 4 attività principali.

5.2.1 Analisi delle ridondanze

Nello schema E-R, una ridondanza è la ripetizione di un'informazione utile ma derivabile da altre, ad esempio:

- Tramite operazioni di interrogazione sulla base di dati.
- Da altri attributi dell'entità stessa. Ad esempio, non ha senso inserire gli attributi `IVA, PrezzoNetto, PrezzoLordo` in un'entità `Prodotto`, in quanto questi attributi sono calcolabili a partire da almeno due di loro.

- Ridondanza derivata da più associazioni sulle stesse entità, ma **solo se le istanze delle associazioni rappresentano lo stesso risultato**.

5.2.2 Eliminazione delle generalizzazioni

Il modello relazionale abbiamo visto essere in grado di rappresentare solo tabelle, non costrutti di ereditarietà come classi e oggetti. Per questo motivo è necessario eliminare tutte le gerarchie rappresentate nel modello concettuale, e ciò può essere fatto in 3 modi differenti:

1. **Accorpamento delle figlie nel padre (Roll-up)**

   - **Cosa si fa:** Si elimina ogni entità figlia. L'entità padre assorbe tutti gli attributi specifici delle entità figlie e tutte le relazioni a cui partecipavano. Viene inoltre introdotto un apposito attributo *selettore* (discriminatore di tipo), ad esempio `TipoPersona = {'Studente', 'Docente'}`, per identificare a quale sottocategoria appartiene ciascuna istanza.
   - **Vantaggi:**
     - Estrema semplicità nella gestione delle proprietà e operazioni comuni.
   - **Svantaggi:**
     - Introduzione di valori nulli (`NULL`): se un'istanza è uno studente, tutti gli attributi propri del docente assumeranno valore nullo.
     - Potenziale spreco di spazio su memoria secondaria e degradazione dell'integrità dei vincoli a livello di colonna.
   - **Condizioni e casi d'uso:** Sempre applicabile senza vincoli di tipo di gerarchia. È la scelta ideale quando le entità figlie possiedono **pochi attributi specifici** e **poche relazioni proprie**, e la maggior parte delle operazioni coinvolge l'insieme complessivo delle istanze (padre e figlie insieme).

2. **Accorpamento del padre nelle figlie (Roll-down)**

   - **Cosa si fa:** Si elimina l'entità padre e si mantengono unicamente le entità figlie. Ciascuna entità figlia eredita tutti gli attributi della classe padre (inclusa la chiave primaria) e tutte le relazioni a cui il padre prendeva parte.
   - **Vantaggi:**
     - Assenza totale di valori nulli dovuti alla gerarchia: ogni relazione contiene unicamente attributi significativi per le sue istanze.
     - Accessi veloci e selettivi per tutte le operazioni mirate a una singola categoria figlia.
   - **Svantaggi:**

  - Ridondanza concettuale: gli attributi e le relazioni originarie del padre vengono replicati nelle specifiche di ogni singola figlia.
- **Condizioni e casi d'uso:** Questo metodo è applicabile in modo naturale **solo se la generalizzazione è TOTALE**. Se fosse parziale, esisterebbero istanze del padre non appartenenti ad alcuna figlia, le quali andrebbero irrimediabilmente perdute eliminando l'entità radice. È inoltre altamente raccomandato che sia **esclusiva** (disgiunta), per evitare di duplicare la stessa entità in più tabelle figlie. Si predilige quando le entità figlie hanno molti attributi distinti e le interrogazioni avvengono quasi sempre separatamente su ciascuna figlia.

3. **Sostituzione della generalizzazione con associazioni**

- **Cosa si fa:** Si mantengono separate sia l'entità padre sia le entità figlie. La gerarchia viene rimpiazzata da associazioni binarie uno a uno (1:1) tra il padre e ciascuna entità figlia. Ogni tabella figlia adotta come chiave primaria la stessa chiave del padre, fungendo contemporaneamente da chiave esterna verso quest'ultimo.
- **Vantaggi:**
  - Massima flessibilità e generalità: è sempre applicabile, indipendentemente dal fatto che la gerarchia sia totale/parziale o esclusiva/sovrapposta.
  - Rispetto rigoroso della semantica originale, assenza di campi nulli anomali e nessuna duplicazione di tuple.
- **Svantaggi:**
  - Impatto sulle prestazioni: per ricostruire l'insieme completo delle informazioni di un'istanza figlia è sempre necessaria una complessa operazione di congiunzione tra le informazioni del padre e quelle della singola classe.
- **Condizioni e casi d'uso:** Sempre applicabile. È la soluzione di riferimento in caso di generalizzazioni **parziali** e/o **sovrapposte**, oppure quando sia l'entità genitore che quelle figlie dispongono di numerosi attributi e relazioni distinti e sussistono accessi eterogenei (sia generalisti che mirati alle sottoclassi).

5.2.3 Partizionamento/accorpamento di entità e associazioni

Si tratta forse dell'operazione più ambigua, in quanto non dipende tanto dall'astrazione che abbiamo fatto del problema, bensì di come devono essere acceduti i dati dall'applicazione. Questa fase può svolgersi secondo 4 diverse modalità a seconda del caso.

1. **Partizionamento verticale di entità**: supponiamo di avere un'entità `Impiegato`, che contiene sia i dati anagrafici che lavorativi. Supponiamo anche che la nostra applicazione debba accedere separatamente ai dati lavorativi e quelli anagrafici. In questo caso potremmo pensare di suddividere l'entità `Impiegato(codice, cognome, indirizzo, data_nascita, livello, stipendio, ritenute)` in due entità separate `DatiAnagrafici(codice,`

`cognome, indirizzo, data_nascita), DatiLavorativi(stipendio, livello, ritenute)`. In seguito queste due entità verranno collegate da un'apposita associazione che le mette in relazione in quanto fanno riferimento allo stesso impiegato.

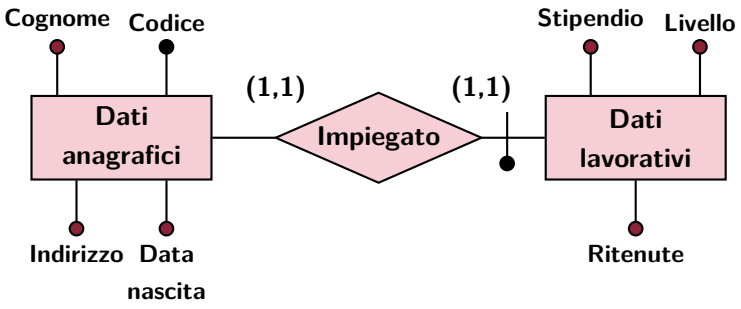
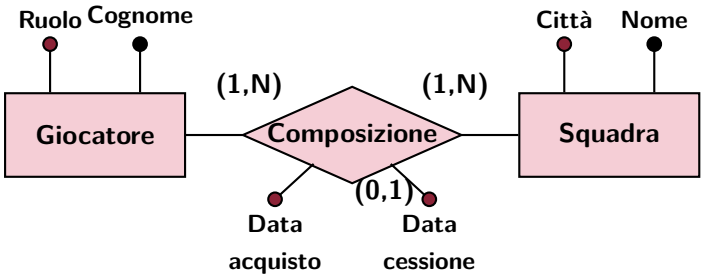


Figura 5.1: Esempio di partizionamento verticale

2. **Partizionamento orizzontale di associazioni**: supponiamo di avere una base di dati che mantiene le informazioni riguardo ai giocatori di una squadra da calcio, sia presenti che passati.



In un caso come questo, supponiamo ci sia richiesto di far accedere separatamente ai dati dei giocatori passati e di quelli correnti. Potremmo dividere dividere l'associazione `Composizione` in due associazioni tra `Composizione attuale` e `Composizione passata`, in modo da gestire separatamente i due casi.

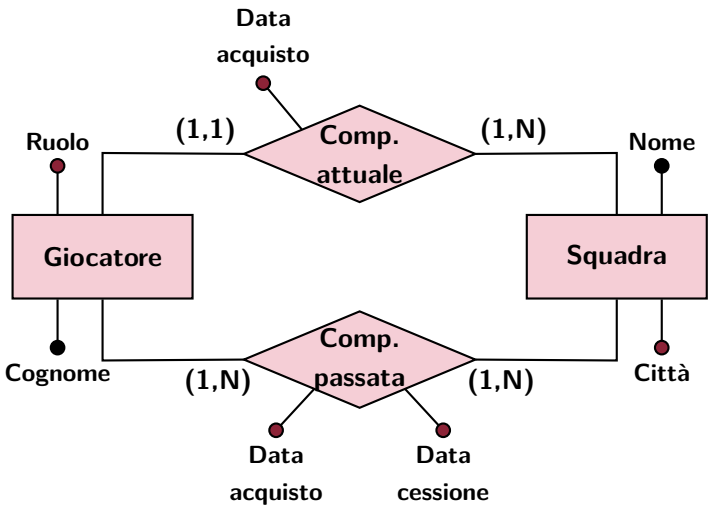
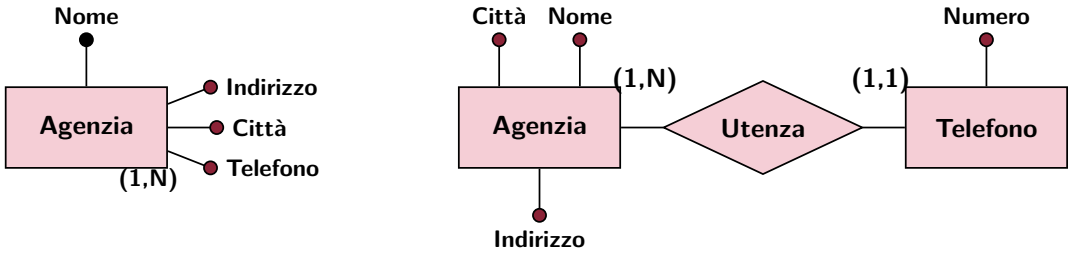


Figura 5.2: Partizionamento orizzontale di associazioni.

3. **Eliminazione di attributi multivalore**. Possiamo avere due istanze di attributi multivalore:

    - Un attributo che ha cardinalità (0, N) oppure (1, N). In tal caso, l'attributo deve diventare un'entità a sé stante, associata all'entità di cui era attributo.



    - In caso di un attributo che comprende più informazioni (come il precedente esempio di `Indirizzo = Via, Numero civico, CAP`), questo deve essere separato nei suoi attributi atomici.

5.2.4 Scelta degli identificatori primari

Come abbiamo visto prima, la **chiave primaria** di un'entità è il più piccolo insieme di attributi che identifica in modo univoco ogni tupla. Guardando con occhio di riguardo anche la progettazione logica, possiamo aggiungere un paio di vincoli:

- Preferibilmente scegliere tra identificatori interno. Se non possibile, è consentito aggiungere un codice artificiale che identifichi la tupla.
- Deve essere un attributo che viene utilizzato nelle operazioni più frequenti ed importanti.

5.3 La traduzione da schema concettuale a logico

Dopo aver rivisto e ristrutturato il modello concettuale con un occhio di riguardo per quella che sarà la rappresentazione dei dati secondo il modello logico, è arrivato il momento di capire come tradurre effettivamente i costrutti del modello concettuale in quelli del modello logico scelto (quello relazionale, nel nostro caso).

5.3.1 Le entità

Di una cosa siamo certi: le entità diventeranno le **tabelle** (o relazioni) del nostro modello logico. Ogni tabella dovrà anche preservare l'insieme degli attributi descritti dal modello concettuale.

5.3.2 Le associazioni

Per le associazioni, ovvero le effettive relazioni che vengono stabilite tra due o più schemi, il discorso è leggermente più delicato, in quanto non esiste un modo strettamente deterministico per rappresentarle.
Prima di discutere separatamente ogni tipo di associazione che possiamo incontrare all'interno dello schema E-R e la rispettiva traduzione nel modello logico, dobbiamo introdurre un fondamentale concetto delle basi di dati relazionali.

Definizione 5.3.1: Vincolo di integrità referenziale

Si tratta di uno dei tanti vincoli di integrità che si stabiliscono all'interno di una base di dati relazionale.
In particolare, questo vincolo stabilisce un **collegamento logico tra due schemi attraverso uno o più attributi con dominio comune**.

Parlando in modo più formale, supponiamo di avere una tabella R_1 con un insieme di attributi X. Prendiamo adesso un'altra tabella R_2. Stabilendo un vincolo di integrità referenziale fra le tabelle R_1 ed R_2 su X significa **imporre** che i valori assunti da X in R_1 debbano comparire come valori della chiave primaria di R_2. Tale insieme di attributi, nella tabella di "arrivo" R_1 viene detto **chiave esterna** o **foreign key**.

In parole povere, stiamo dicendo che se in R_1 abbiamo un insieme di attributi X, affinché il vincolo di integrità referenziale con R_2 sia valido dobbiamo avere la certezza che R_2 contenga i valori di chiave primaria espressi da X in almeno una n-upla.

Avendo definito il concetto di chiave esterna, possiamo procedere ad analizzare uno ad uno i tipi di associazione.

5.3.2.1 Associazioni: Molti a molti

Un'associazione *molti a molti* si traduce come una vera e propria **tabella** che ha come chiave **l'insieme degli attributi chiave delle tabelle coinvolte nell'associazione**. Inoltre, tale tabella include anche eventuali attributi dell'associazione.

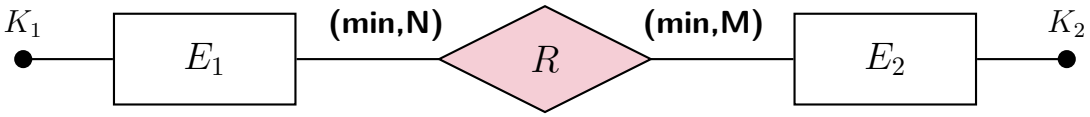


Figura 5.3: Modello concettuale generico di una associazione molti a molti.

Secondo quanto detto prima, il frammento di schema mostrato si traduce con le seguenti relazioni:

E1(**K1**, attributi di E1...)

E2(**K2**, attributi di E2...)
R(**K1**, **K2**, attributi dell'associazione...)

E stabilendo anche i vincoli di integrità referenziale:

- Tra K1 in R e K1 in E1.
- Tra K2 in R e K2 in E2.

Generalmente, è consigliato rinominare gli attributi che costituiscono un vincolo di integrità referenziale, dando come nome un qualcosa di indicativo della tabella di provenienza (es: se l'attributo di partenza si chiama `Codice` e proviene dalla tabella `Impiegato`, potremmo chiamare la chiave esterna come `FK_Codice_Impiegato` oppure semplicemente `Impiegato`. In caso di chiavi composte da più attributi, possiamo rinominarle come `Impiegato1`, `Impiegato2`, ...).

Particolare attenzione deve essere posta sulle associazioni molti a molti ricorsive, in quanto in quel caso è di vitale importanza il nome delle chiavi esterne.

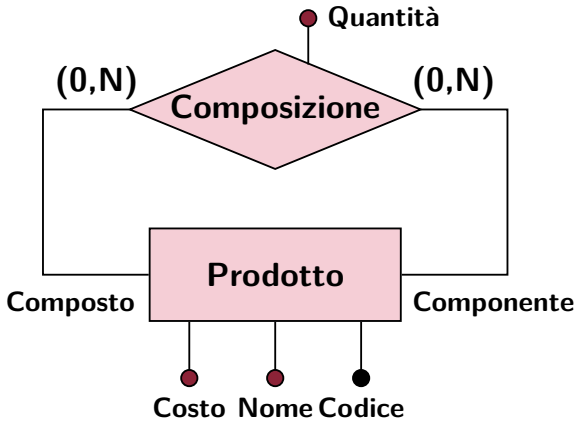


Figura 5.4: Esempio di associazione molti a molti ricorsiva.

La derivazione di questo modello concettuale sarà

Prodotto(**Codice**, Nome, Costo)
Composizione(**CodiceComposto**, **CodiceComponente**, Quantita)

In questo caso i due codici presenti nella tabella `Composizione` non saranno altro che codici di prodotti però inseriti come chiavi esterne.

Un ultimo esempio sulle associazioni molti-a-molti per associazioni n-arie:

In questo caso si deriva come ci si aspetterebbe, ma si è scelto di mostrare un esempio per evitare che il lettore si cogliesse impreparato. Il fulcro è sempre creare una tabella per l'associazione e mettere al suo interno aventuali attributi dell'associazione e, come chiave primaria, l'insieme

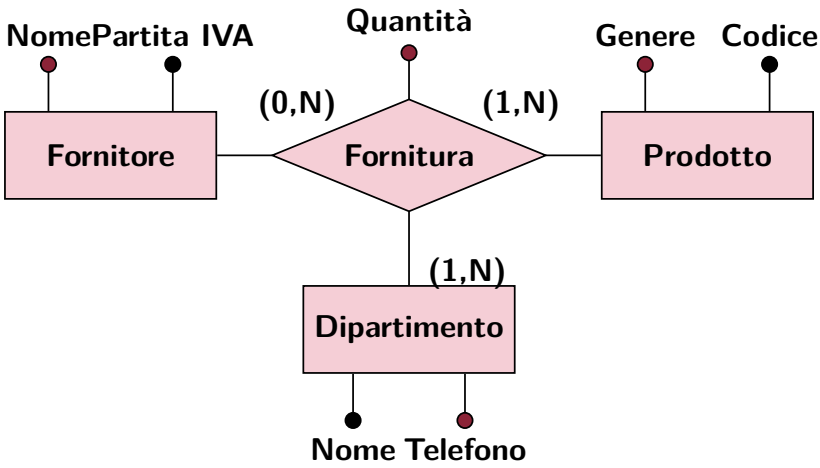


Figura 5.5: Esempio di associazione n-aria molti-a-molti.

delle chiavi di ogni tabella coinvolta.
La derivazione risulta dunque:

Fornitore(**PartitaIVA**, Nome)
Prodotto(**Codice**, Genere)
Dipartimento(**Nome**, Telefono)
Fornitura(**Fornitore**, **Prodotto**, **Dipartimento**, Quantita)

5.3.2.2 Associazioni: Uno a molti

Per questo tipo di associazioni, si può procedere in due modalità diverse:

1. **Creare una tabella per l'associazione**, al cui interno si metterà:
   - Come **chiave primaria**, la **chiave primaria della tabella che partecipa all'associazione con cardinalità (min,1)** (che sarà anche chiave esterna, legata da vincolo di integrità referenziale).
   - Come **chiave esterna**, la **chiave primaria della tabella che partecipa all'associazione con cardinalità (min,N)**.
2. **Inserire nella tabella che partecipa con cardinalità (min,1) un vincolo di integrità referenziale alla chiave primaria della tabella che partecipa con (min, N)**. Tutti gli eventuali attributi dell'associazione andranno a finire nella tabella con cardinalità (min,1).

Vediamo di seguito un esempio di modello concettuale, e discutiamo le due possibili traduzioni nel modello logico.

Seguendo il primo modo descritto, ovvero quello di creare una nuova tabella, la derivazione sarà la seguente:

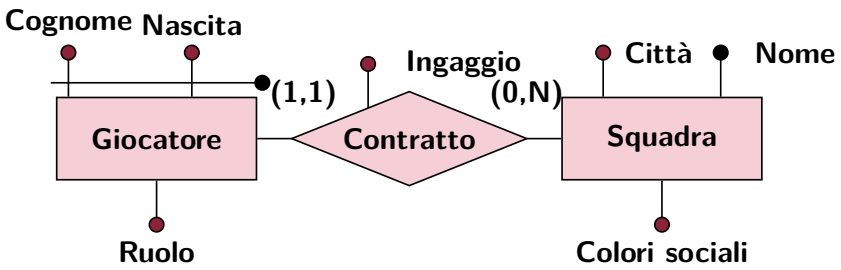


Figura 5.6: Esempio di associazione uno-a-molti tra due entità

Giocatore(**Cognome**, **Nascita**, Ruolo)
Squadra(**Nome**, ColoriSociali, Citta)
Contratto(**Cognome**, **Nascita**, *Squadra*, Ingaggio)

Il **vantaggio** di questa rappresentazione è che, creando una tabella apposita per la relazione tra le due tabelle, *non dovremmo preoccuparci di eventuali valori nulli o di istanze delle tabelle che non hanno corrispondenza nell'associazione*. Il principale svantaggio consiste invece nel fatto che stiamo definendo un'ulteriore schema per la nostra relazione, dunque un ulteriore tabella da indicizzare per il DBMS (parleremo dell'efficienza più avanti nel corso).

Se invece volessimo derivare il precedente modello nel secondo modo, otterremmo:

Squadra(**Nome**, Citta, ColoriSociali)
Giocatore(**Nome**, **Nascita**, Ruolo, *Squadra*, Ingaggio)

In questa rappresentazione, all'interno della tabella `Giocatore` (quella che partecipa con cardinalità (1,1) all'associazione) mettiamo un vincolo di chiave esterna alla chiave primaria della tabella `Squadra` e l'attributo dell'associazione `Ingaggio`.

Questo metodo rappresenta la soluzione più semplice e compatta, in quanto non richiede la creazione di tabelle aggiuntive ma si limita ad aggiungere attributi ad una già esistente. Lo svantaggio è legato alla possibile presenza di valori nulli all'interno dell'entità *debole*: se questa partecipasse all'associazione con una cardinalità (0,1) significherebbe che ci potrebbero essere istanze della tabella che non hanno correlazione con quella associata. Per tabelle con un gran numero di record, si potrebbe verificare un ingente spreco di memoria.

Definizione 5.3.2: Entità debole

Un'entità (o successivamente una tabella nel modello relazionale) si definisce **debole** quando **la sua esistenza dipende dall'esistenza di una o più entità**.
In genere, sono considerate deboli quelle entità al cui interno sono presenti chiavi esterne di un'altra entità, in quanto per essere definite devono prima essere definite le entità da cui provengono tali chiavi esterne.

Andiamo a vedere anche per le associazioni uno-a-molti le casistiche di ricorsività oppure n entità coinvolte.

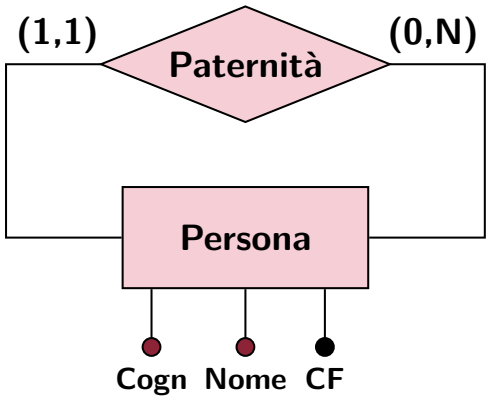


Figura 5.7: Associazione uno-a-molti ricorsiva.

Nel caso di questa associazione, si vogliono spendere due parole su entrambe le traduzioni in quanto potrebbero presentare delle criticità:

1. **Creazione di una tabella per la relazione**, e dunque si avrà lo schema

   Persona(**CF**, Nome, Cogn)
   Paternita(**Figlio**, *Padre*)

   Si presti particolare attenzione alla scelta della chiave primaria nella tabella `Paternita`:

   - Se avessimo scelto solo **Padre** ci saremmo trovati nella situazione in cui un padre può avere al più un figlio, chiaramente errata per lo schema rappresentato.
   - Se avessimo scelto come chiave la coppia **Figlio, Padre** ci saremmo potuti trovare nella situazione in cui un figlio è associato a più di un padre (in quanto, fissando il figlio e modificando il padre si possono generare tuple differenti).

2. **Unificazione in una sola tabella**. In questo caso lo schema sarebbe stato semplicemente Persona(**CF**, Nome, Cogn, *Padre*), dove *Padre* rappresenta una chiave esterna.

   Si noti una criticità di questa implementazione: per come definito prima, il vincolo di integrità referenziale impone che i valori associati alla chiave esterna siano valori **esistenti** nella tabella di partenza. Siccome in questo caso l'associazione è ricorsiva, la tabella di partenza è la stessa di quella di arrivo. Dunque questo ci mette nella condizione in cui ci è sostanzialmente **impossibile** (almeno all'inizio) inserire delle persone all'interno della tabella, in quanto non potremmo associarle un padre.
   Ci sono due possibili soluzioni:

   (a) Disabilitare temporaneamente i controlli di integrità del DBMS, per permettere di inserire delle persone e successivamente riabilitarli per creare la corrispondenza effettiva.

(b) Impostare il valore della chiave esterna come non obbligatorio (**RISCHIOSO!!!**) e inserire il valore della chiave esterna solo *dopo aver effettuato sufficienti inserimenti*.

Vediamo di seguito un esempio anche con le associazioni n-arie molti-a-molti.

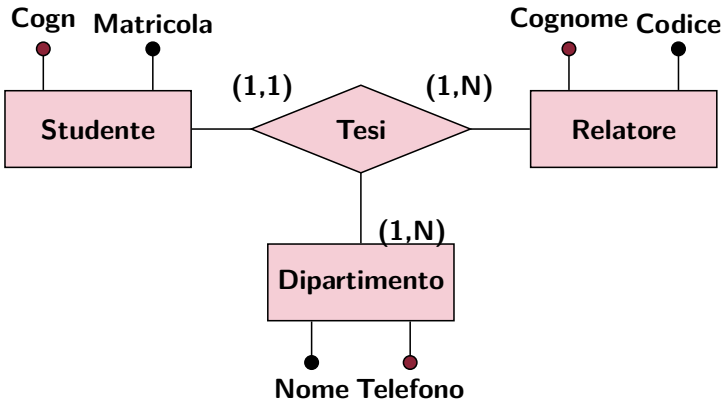


Figura 5.8: Associazione ternaria molti-a-molti.

Il trucco per le associazioni n-arie è *leggerle a due a due*, dunque prendendo una coppia di entità e leggere l'associazione tra queste. Fare poi questo lavoro per tutte le entità coinvolte. Anche in questo caso la traduzione può avvenire in due modi:

1. **Creando la tabella per l'associazione**, ottenendo il seguente schema della base di dati:

   Studente(**Matricola**, Cogn)
   Relatore(**Codice**, Cogn)
   Dipartimento(**Nome**, Telefono)
   Tesi(**Studente**, *Relatore*, *Dipartimento*)

   Ci accorgiamo però che le tabelle `Studente` e `Tesi` hanno la stessa chiave primaria, e dunque conviene unificarle.

2. **Unificare tutto in un'unica tabella**, ottenendo lo schema:

   Relatore(**Codice**, Cogn)
   Dipartimento(**Nome**, Telefono)
   Studente(**Matricola**, Cogn, *Relatore*, *Dipartimento*)

5.3.3 I valori NULL

I valori NULLI rappresentano mancanza di informazione oppure che un'informazione non è applicabile a quella n-upla. Vediamo un esempio con un'informazione che usiamo tutti i giorni:

il numero di telefono.
In una base di dati, il numero di telefono di una persona potrebbe assumere un valore nullo in diverse circostanze:

- La persona effettivamente non possiede un telefono cellulare e dunque neanche un numero.
- Non sappiamo se la persona abbia un numero di telefono.
- La persona ha un numero di telefono ma non lo conosciamo.

Nell'ottica della progettazione logica, dobbiamo stabilire questo vincolo (indicando che l'attributo può assumere valori NULL), ma in generale si tende a non utilizzare questa pratica, in quanto per la definizione di tupla questa deve possedere un valore per ogni attributo nello schema. Infatti, una possibile tecnica potrebbe essere quella di attribuire un valore di **default**.
Ci rendiamo anche conto di come molti campi debbano categoricamente essere **obbligatori**. Si tratta solitamente dei campi che contengono informazioni fondamentali per l'identificazione della tupla (come ad esempio la matricola di uno studente, che ci permette di identificarlo univocamente) oppure il voto di uno studente ad un appello di esame (non usiamo questa informazione per identificare lo studente, ma si tratta di un'informazione fondamentale in una base di dati universitaria).

Entrando più nello specifico del valore NULL, possiamo dire che:

- Si tratta di un valore **polimorfo**, cioè che non appartiene nello specifico a nessun dominio, ma può sostituire valori in qualsiasi dominio (ad esempio, se abbiamo degli attributi di tipo intero e stringhe, possiamo comunque inserire NULL come valore in quanto sarà compatibile con entrambi).
- Due valori NULL, anche se sullo stesso dominio, sono considerati **diversi**.
- **IMPORTANTE**! NULL $\neq 0$.

5.4 Vincoli di integrità

Definizione 5.4.1: Vincolo di integrità

Un **vincolo di integrità** è una proprietà che deve essere categoricamente rispettata da **ogni istanza** della base di dati, in quanto è strettamente legata allo schema di una relazione.

Un'istanza della base di dati si dice **corretta** se soddisfa tutti i vincoli di integrità associati al suo schema.

I vincoli di integrità si possono suddividere in due tipologie:

Vincoli di integrità intra-relazionali: sono **vincoli che si stabiliscono sugli attributi o sulle tuple di una singola relazione**. Alcuni esempi di questi sono:

- Vincolo di chiave primaria: attributo obbligatorio e mai nullo. Identifica univocamente una tupla all'interno della relazione.
- Vincolo di dominio: quando si restringe il dominio di un particolare attributo.
- Vincolo di unicità: stabilisce che il valore di un attributo in una tupla t deve essere univoco e non preso da nessun'altra tupla.
- Vincolo di obbligatorietà: stabilisce che un attributo deve necessariamente avere un valore.

Vincoli di integrità inter-relazionali: sono **vincoli che si stabiliscono tra relazioni diverse**, "collegando" ad esempio attributi con lo stesso dominio e creando un legame in base al valore assunto dai due attributi. Un esempio è il vincolo di chiave esterna che abbiamo citato qualche sezione fa.

Facciamo alcuni esempi concreti di violazione di vincoli, oppure di casi in cui si potrebbero imporre dei vincoli.

CodiceImp	Cognome	Ruolo	Assunzione	Dip
COD1	Rossi	Analista	1795	01
COD2	Bianchi	Analista	1990	05
COD2	Neri	Amministratore	1985	01

CodiceDip	Nome
01	Progettazione
02	Amministrazione

Considerando quest'esempio di base di dati, ci sono molti problemi: vediamo dove risiedono.

1. L'impiegato `COD1` risulta avere anno di assunzione 1795. Ciò è chiaramente impossibile, e una possibile soluzione potrebbe essere stabilire un anno di assunzione minimo (*vincolo di dominio*).
2. L'impiegato "Bianchi" risulta appartenere al dipartimento di codice "05", che è tuttavia inesistente, in quanto non presente in alcuna tupla della tabella `Dipartimenti`. Questo si

potrebbe risolvere stabilendo un *vincolo di chiave esterna* tra l'attributo `Dip` di `Impiegato` e l'attributo `CodiceDip` di `Dipartimenti`.

3. Vi è una duplicazione del codice identificativo per gli impiegati "Bianchi" e "Neri". Ciò si potrebbe risolvere imponendo un *vincolo di unicità* sui valori della colonna `CodiceImp`.

Ovviamente esistono tanti altri tipi di vincoli, e altrettanti modi per definirli nello schema della base di dati, ma li vedremo più nel dettaglio durante la creazione effettiva delle relazioni.

6

Algebra relazionale

L'algebra relazionale è un **linguaggio formale che si basa su modelli matematici utile per interrogare una base di dati relazionale**. Consiste in un insieme di operatori che agiscono su una singola relazione, detti appunto **unari**, e un insieme di attributi che agiscono su coppie di relazioni, per l'appunto **binari**. Il **risultato** di ognuna di queste operazioni è una **nuova relazione**.

Osservazione 6.0.1: Algebra relazionale come linguaggio procedurale

L'algebra relazionale può essere anche vista come **linguaggio procedurale**, ovvero un linguaggio per cui si **esprimono in sequenza le operazioni da eseguire, come operare sulle relazioni e cosa estrarre da esse**.
Questa è un'importante differenza con il linguaggio DQL con il quale effettueremo effettivamente le interrogazioni, che è invece un linguaggio **dichiarativo**, cioè dove noi non diciamo cosa fare al linguaggio ma solo cosa vogliamo ottenere (sarà poi il DBMS ad attuare le dovute operazioni per mostrarci il risultato richiesto).

Di seguito elenchiamo i vari operatori dell'algebra relazionale, discutendoli poi più nel dettaglio in sezioni separate.

6.1 Proiezione

Si tratta di un **operatore unario che permette di selezionare da una relazione tutte le tuple limitando ciascuna ad un insieme di attributi specificato**.
Formalmente parlando, data un'istanza di relazione r con un insieme di attributi X, se effettuiamo una proiezione su r sull'insieme $\{A_1, A_2, \ldots, A_k\} \subseteq X$ otteniamo una nuova relazione che contiene tutte le tuple $t \in r$ ognuna ristretta agli attributi $\{A_1, A_2, \ldots, A_k\}$.
Corrisponde all'operatore `SELECT DISTINCT` del linguaggio DQL, che permette di specificare quali attributi della relazione vogliamo visionare come risultato della query.

L'operazione di proiezione si indica con la notazione

$$\pi_{A_1,A_2,\ldots,A_k}(r)$$

Esempio 6.1.1: Esempio di proiezione su una tabella "Clienti"

Nome	C#	Citta
Rossi	C1	Roma
Rossi	C2	Milano
Bianchi	C3	Roma
Verdi	C4	Roma

Supponiamo di voler sapere tutti i nomi dei clienti:

$$\pi_{\text{Nome}}(\text{Clienti})$$

La relazione risultante sarà

Rossi
Bianchi
Verdi

Ovviamente, dato che il risultato dell'operazione è a sua volta una relazione (di fatto un sottoinsieme), nella stessa colonna non potremo avere valori duplicati, ed ecco perché appare solo 1 volta "Rossi".

Osservazione 6.1.1: Come ottenere tutte le tuple

Come abbiamo visto, la proiezione è un'operazione insiemistica, e per tanto non mantiene duplicati. Se volessimo fare in modo che la relazione risultante dalla proiezione mostrasse tutte le tuple, anche quelle contenenti omonimi (ma questo discorso vale per qualsiasi istanza in cui si va ad usare un'operazione di proiezione) dovremmo includere anche la **chiave primaria** della relazione nell'insieme di attributi della proiezione.

Questo avviene in quanto ricordiamo essere la chiave primaria di una tupla ad identificarla univocamente all'interno dell'istanza di relazione. Ed è proprio il motivo per cui quando

inseriamo come unico attributo della selezione il campo "Nome", essendo quello l'unico diventerà lui la chiave primaria della nuova relazione, e dunque non saranno ammesse occorrenze duplicate.

Possiamo dire che la proiezione compie un **taglio verticale** su una relazione, eliminando solo alcune colonne (e i relativi valori per ogni tupla) dalla relazione di partenza.

6.2 Selezione

Si tratta di un **operatore unario** che permette di **selezionare dalla relazione di partenza solo le tuple che rispettano una determinata condizione** C.
Formalmente parlando, sia r un istanza di relazione e C una condizione proposizionale. L'operazione di selezione produce una nuova relazione in cui sono contenute tutte le tuple $t \in r$ per cui C risulta vera.
Quest'operazione è il corrispettivo della clausola `WHERE` nel linguaggio DQL. L'operazione di selezione si indica con la notazione

$$\sigma_C(r)$$

Esempio 6.2.1: Esempio di selezione su una tabella "Clienti"

Nome	C#	Città	.....
Rossi	C1	Roma	.....
Rossi	C2	Milano	.....
Rossi	C3	Roma	.....
Bianchi	C4	Roma	.....
Verdi	C5	Roma	.....

Nome	C#	Città	.....
Rossi	C1	Roma	.....
Rossi	C3	Roma	.....
Bianchi	C4	Roma	.....
Verdi	C5	Roma	.....

La relazione di destra è la relazione risultante dall'operazione $\sigma_{\text{Citta} = \text{"Roma"}}(\text{Clienti})$.

6.3 Unione

L'operatore di unione è un operatore binario che ci permette di costruire una nuova relazione che conterrà **tutte le tuple che appartengono ad entrambe le relazioni** (dunque tutte le tuple della prima e tutte le tuple della seconda).
La definizione formale dell'operatore è proprio quella matematiche che ci aspetteremmo: date due istanze di relazione r_1, r_2, l'unione tra queste due relazioni è l'insieme $\{t \mid t \in r_1 \vee t \in r_2\}$.

Definizione 6.3.1: Relazioni "union compatibili"

L'operazione di unione non è applicabile su qualsiasi coppia di relazioni. Due relazioni si dicono **union compatibili** se:

- **Hanno lo stesso numero di attributi**.
- **Hanno gli stessi domini** (in ordine).

L'operazione è ugualmente chiamata `UNION` anche nel linguaggio DQL, e si indica con la seguente notazione:

$$r_1 \cup r_2$$

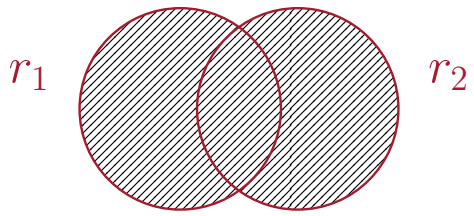


Figura 6.1: Rappresentazione grafica dell'unione tra r_1 e r_2.

Di seguito un esempio di come risulta essere l'unione tra due relazioni, rispettivamente `Docenti` e `Amministrativi`.

Esempio 6.3.1: Esempio di unione tra due relazioni

Nome	CodDoc	Dipartimento
Rossi	D1	Matematica
Rossi	D2	Lettere
Bianchi	D3	Matematica
Verdi	D4	Lingue

Nome	CodAmm	Dipartimento
Esposito	A1	Lingue
Riccio	A2	Matematica
Pierro	A3	Lettere
Verdi	A4	Lingue
Bianchi	A5	Lingue

L'unione tra queste due relazioni risulta essere la seguente relazione:

Nome	Cod	Dipartimento
Rossi	D1	Matematica
Rossi	D2	Lettere

Bianchi	D3	Matematica
Verdi	D4	Lingue
Esposito	A1	Lingue
Riccio	A2	Matematica
Pierro	A3	Lettere
Verdi	A4	Lingue
Bianchi	A5	Lingue

Osservazione 6.3.1: Coerenza degli attributi nell'unione

E' doveroso fare un'osservazione. Due relazioni definite rispettivamente come `Studenti(Nome, Eta)` e `Professore(Nome, NumeroTelefono)` sono di fatto union compatibili: il campo `Nome` è in entrambi i casi un attributo di dominio `String`, e gli attributi `Eta` e `NumeroTelefono` sono entrambi numeri interi. Ci rendiamo conto che però il significato dei due attributi sia diverso, nonostante condividano lo stesso dominio. Dunque la relazione risultante sarà ambigua o incoerente nel campo di dominio `Intero`.

Osservazione 6.3.2: Come rendere due relazioni Union compatibili

Nome	CodDoc	Dipartimento
Rossi	D1	Matematica
Rossi	D2	Lettere
Bianchi	D3	Matematica
Verdi	D4	Lingue

Nome	CodAmm	Dipartimento	Stip
Esposito	A1	Lingue	1250
Riccio	A2	Matematica	2000
Pierro	A3	Lettere	1000

Ora come ora, queste relazioni non risultano essere Union compatibili, in quanto la relazione di destra presenta un attributo in più. Se però prendessimo come relazione quella risultante da $\pi_{\text{Nome, CodAmm, Dipartimento}}(\text{Amministrativi})$ otterremmo una nuova relazione che sarà adesso Union compatibile con quella di sinistra.

Il succo della questione è: **possiamo rendere due relazioni Union compatibili considerando la relazione risultante dalla proiezione di una di queste**. Ovviamente supponendo che abbiano almeno un dominio comune, altrimenti ciò rimarrebbe impossibile.

6.4 Differenza

Si tratta di un operatore **binario** che si può applicare tra due relazioni che sono già "Union compatibili". Il risultato dell'operazione è una **relazione che contiene tutti gli elementi della prima relazione che non sono anche nella seconda**.
Definendo quest'operazione matematicamente, abbiamo che la differenza tra una relazione r_1, r_2 è l'insieme $\{t \mid t \in r_1 wedget \notin r_2\}$.
Tale operazione si indica nel seguente modo

$$r_1 - r_2$$

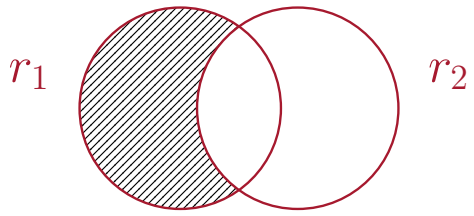


Figura 6.2: Differenza insiemistica tra le relazioni r_1 e r_2 $(r_1 - r_2)$.

Esempio 6.4.1: Esempio di differenza tra due relazioni

Consideriamo le seguenti relazioni, `Studenti` e `Amministrativi`:

Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Matematica
Verdi	C4	Lingue

Nome	CodFiscale	Dipartimento
Esposito	C5	Lettere
Riccio	C6	Matematica
Pierro	C7	Lingue
Bianchi	C3	Matematica

La differenza tra relazioni **non è un'operazione commutativa**. Dunque fare `Studenti - Amministrativi` è diverso dal fare `Amministrativi - Studenti`.
Di seguito vediamo entrambi i risultati:

Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Verdi	C4	Lingue

Nome	CodFiscale	Dipartimento
Esposito	C5	Lettere
Riccio	C6	Matematica
Pierro	C7	Lingue

A sinistra abbiamo la relazione `Studenti - Amministrativi`, che contiene al suo interno *tutti gli studenti tranne quelli che sono anche amministrativi.*
A destra abbiamo la relazione `Amministrativi - Studenti` che contiene *tutti gli amministrativi tranne quelli che sono anche studenti.*

6.5 Intersezione

Per quanto si potrebbe pensare all'intersezione come un operatore primitivo, esso è in realtà un operatore derivato dalla seguente operazione insiemistica: $r_1 - (r_1 - r_2)$.
Il risultato dell'intersezione tra due relazioni è una nuova relazione che contiene **solo le tuple comuni ad entrambe le relazioni**. Formalmente la definiremmo come $\{t \mid t \in r_1 \wedge t \in r_2\}$.
L'operazione di intersezione tra due relazioni si denota nel seguente modo:

$$r_1 \cap r_2$$

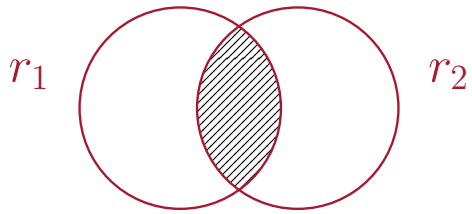


Figura 6.3: Intersezione tra le relazioni r_1 e r_2 ($r_1 \cap r_2$).

Esempio 6.5.1: Esempio di intersezione tra due relazioni

Consideriamo le precedenti relazioni `Studenti` e `Amministrativi`:

Nome	CodFiscale	Dipartimento
Rossi	C1	Matematica
Rossi	C2	Lettere
Bianchi	C3	Matematica
Verdi	C4	Lingue

Nome	CodFiscale	Dipartimento
Esposito	C5	Lettere
Riccio	C6	Matematica
Pierro	C7	Lingue
Bianchi	C3	Matematica

L'intersezione tra relazioni è ovviamente un'operazione commutativa, in quanto prende i record che si trovano in entrambe le relazioni. Il risultato dunque dell'intersezione delle precedenti relazioni sarà dunque l'unico studente che è sia `Studente` che `Amministrativo`.

Nome	CodFiscale	Dipartimento
Bianchi	C3	Matematica

6.6 Prodotto cartesiano

L'operazione di prodotto cartesiano tra due istanze di relazione funziona esattamente come il prodotto cartesiano tra due insiemi: si ottengono tutte le n-uple della prima relazione concatenate a quelle della seconda relazione. Abbiamo già parlato di prodotto cartesiano quando definivamo il concetto stesso di *relazione*, dunque ci limiteremo solo a dire che il prodotto cartesiano tra una relazione r_1 e una relazione r_2 si indica nel seguente modo:

$$r_1 \times r_2$$

Vedendo un esempio più concreto, supponiamo di avere una tabella `Docenti` e una tabella `Corsi`.

ID_Docente	Cognome	CodCorso
D1	De Marsico	C01
D2	Pirri	C03

CodCorso	NomeCorso
C01	Basi di dati
C02	Algoritmi
C03	Reti

Il prodotto cartesiano tra queste due tabelle risulta essere il seguente:

ID_Docente	Cognome	CodCorso	NomeCorso
D1	De Marsico	C01	Basi di Dati
D1	De Marsico	C02	Algoritmi
D1	De Marsico	C03	Reti
D2	Pirri	C01	Basi di Dati
D2	Pirri	C02	Algoritmi
D2	Pirri	C03	Reti

Come vediamo, dal prodotto cartesiano di queste due tabella abbiamo ottenuto una combinazione di tutte le istanze di docente con tutte le possibili istanze di corso. Tuttavia, come vediamo, avendo attributi con lo stesso nome, come riusciamo a determinare quale attributo è associato a quale relazione? E come facciamo a capire quale corrispondenza risulta veritiera?

6.7 Ridenominazione

A tal proposito, introduciamo l'operatore di **ridenominazione**, che è l'unico che agisce sullo schema della relazione. Il punto di questo operatore è proprio modificare il nome della relazione e degli attributi al suo interno, proprio per evitare casi del genere in cui vi è presente un attributo che nelle relazioni di partenza ha lo stesso nome.
L'operazione di ridenominazione ha la seguente sintassi:

$$\text{NuovoNomeRelazione} = \rho_{\text{NuovoNome1, NuovoNome2, ...} \leftarrow \text{Nome1, Nome2, ...}}(\text{R})$$

dove "NuovoNome(i)" indica il nuovo nome che si vuole dare agli attributi nella nuova relazione.

Dunque per porre un rimedio alla situazione precedente, si potrebbe ad esempio cambiare il nome dell'attributo che si riferisce al corso nella tabella `Docenti`, ad esempio nel seguente modo: $\text{DocentiNuovo} = \rho_{\text{CodCorsoInsegnato <- CodCorso(R)}}$. Avendo ridenominato l'attributo `CodCorso` della tabella `Docenti` possiamo effettuare il prodotto cartesiano senza il rischio di perdita di informazioni.
Effettuando il prodotto cartesiano, il risultato sarà adesso:

ID_Docente	Cognome	CodCorsoInsegnato	CodCorso	NomeCorso
D1	De Marsico	C01	C01	Basi di Dati
D1	De Marsico	C01	C02	Algoritmi
D1	De Marsico	C01	C03	Reti
D2	Pirri	C03	C01	Basi di Dati
D2	Pirri	C03	C02	Algoritmi
D2	Pirri	C03	C03	Reti

Notiamo però adesso che c'è una discrepanza di valori: in alcune righe, il codice del corso insegnato dal docente non risulta essere lo stesso di quello corrispondente nella tupla. Basta ad esempio vedere la seconda e la terza tupla: per la tabella docenti, noi sappiamo che il docente di codice `D1` insegna il corso di codice `C01`; tuttavia nelle righe 2 e 3 il docente `D1` risulta avere una corrispondenza con i corsi di codice `C02` e `C03`.

Corollario 6.7.1: Correttezza di una query che coinvolge più tabelle

Quando enunciato di seguito deve diventare un mantra per tutte le volte che si dovrà scrivere una query in cui vengono coinvolte più tabelle: se queste hanno un dominio comune che rappresenta la stessa informazione (come `CodCorso` nell'esempio antecedente)

è *strettamente necessario* **eguagliare quei due valori in modo da mantenere all'interno della tabella risultante solo i record che contengono la corrispondenza corretta**.

Dunque, nell'esempio precedente, se volessimo ottenere la tabella contenente solo le informazioni coerenti, l'operazione da fare dovrà essere una selezione delle righe in cui `CodCorsoInsegnato = CodCorso`.

$$\sigma_{\text{CodCorsoInsegnato} = \text{CodCorso}}(\text{Docenti} \times \text{Corsi})$$

Ottenendo dunque la seguente relazione:

ID_Docente	Cognome	CodCorsoInsegnato	CodCorso	NomeCorso
D1	De Marsico	C01	C01	Basi di Dati
D2	Pirri	C03	C03	Reti

Dove, come vediamo, abbiamo la duplicazione dell'attributo codice corso (seppur con nome diverso, il valore di quel campo è sostanzialmente duplicato). Seppur questo non sia un errore, potremmo decidere di omettere uno dei due attributi per garantire un risultato più carino. Si può ottenere con una semplice proiezione:

$$\pi_{\text{ID_Docente, Cognome, CodCorsoInsegnato, NomeCorso}}(\sigma_{\text{CodCorsoInsegnato} = \text{CodCorso}}(\text{Docenti} \times \text{Corsi}))$$

Ottenendo così infine la tabella

ID_Docente	Cognome	CodCorsoInsegnato	NomeCorso
D1	De Marsico	C01	Basi di Dati
D2	Pirri	C03	Reti

6.8 Join naturale

Fatte tutte queste considerazioni sul prodotto cartesiano, sulla necessità di ridenominare gli attributi e sulla stretta necessità di eguagliare due domini che rappresentano la stessa informazione per garantire la coerenza dei dati, introduciamo l'operatore dell'algebra relazionale che accorpa tutte queste operazioni: il **join naturale**.
L'operazione di join naturale funziona esattamente come tutto il processo visto precedentemente:

1. Fa il prodotto cartesiano tra due relazioni.

2. Di questo prodotto cartesiano, se specifichiamo due domini comuni che condividono la stessa informazione, verranno selezionate solo le tuple che hanno valore uguale in quei due attributi.

Tale operazione è infatti definita come segue:

$$r_1 \bowtie r_2 := \pi_{XY}(\sigma_C(r_1 \times r_2))$$

Osservazione 6.8.1: Funzionamento del Join naturale

L'operazione di join naturale funziona ad una sola condizione: che i **due attributi condividenti dominio e "scope" informativo abbiano lo stesso nome in entrambe le due relazioni**. Dunque nell'esempio precedente non sarebbe servito a nulla usare il join naturale, a meno che noi non avessimo usato la ridenominazione per rendere i nomi degli attributi uguali.

Nota che il risultato di un'operazione di Join naturale potrebbe tranquillamente essere una **relazione vuota**, in caso di **mancate corrispondenze tra i valori delle tuple**.

Osservazione 6.8.2: Importanza dell'ordine delle operazioni

Si nota come l'ordine in cui vengono eseguite i vari operatori in un'interrogazione complessa è fondamentale, in quanto da ciò potrebbe cambiare l'efficienza dell'interrogazione. Ovviamente tutti i DBMS relazionali moderni implementano già le interrogazioni eseguendo le operazioni nell'ordine più efficiente. Tuttavia è importante notare come, ad esempio, effettuare una selezione delle tuple compatibili **prima** di fare il prodotto cartesiano sia un risparmio particolarmente sostanzioso, in quanto si riduce il numero di tuple che vanno combinate l'un l'altra.

6.9 Theta Join

L'operazione di Join naturale è un'operazione potentissima, che ci permette finalmente di costruire gli effettivi collegamenti mediante valori tra le relazioni. Tuttavia presenta alcune limitazioni:

- I due attributi che condividono il dominio devono avere lo stesso nome (risolvibile con la ridenominazione ma è un'operazione costosa).
- Gli attributi omonimi possono esclusivamente essere uguagliati e non confrontati in alcun altro modo.
- Elimina le colonne duplicate, dunque di quelle che consentono di creare la corrispondenza ne mantiene una sola.

A tal proposito, si è deciso di definire un operatore chiamato θ-Join (Theta Join) che rifinisce tali aspetti:

- Non è necessario che gli attributi colleganti abbiano lo stesso nome.
- Il confronto tra questi attributi è definibile sotto qualsiasi operatore matematico di confronto.
- Mantiene tutte le colonne delle relazioni di partenza, dunque non applica alcuna proiezione sul risultato finale.

La definizione formale di tale operazione è la seguente:

$$r_1 \bowtie_{A\theta B} r_2 := \sigma_{A\theta B}(r_1 \times r_2)$$

6.10 Quantificatori unari e globali

Finora tutte le operazioni di interrogazione che abbiamo definito e svolto sulle nostre relazioni si sono basate su delle richieste molto simili: *trovare tutte le istanze di ... tali che esiste almeno un ...*
Detto così è un po' astratto, però con un esempio forse si chiarirà il concetto: *trovare tutti gli studenti che hanno superato almeno un esame.* Come vediamo, ci viene chiesto SE ESISTONO gli studenti con almeno un esame, e memorizzarli in una relazione. Dunque sappiamo come operare: scorriamo tutte le tuple di una relazione e appena troviamo una tupla con `Superato = True`, allora prendiamo codice e nome di quello studente e li inseriamo come tupla nella nuova relazione.

Insomma, finora tutte le nostre interrogazioni si sono basate sulla costante logica di **scansionare una per una le tuple e decidere lì sul momento se quella tupla fosse compatibile o meno**. Matematicamente, questo equivale al quantificatore essenziale $\exists$.

Vogliamo ora spostare il nostro ragionamento: e se ci fosse stato chiesto *tutti gli studenti che non hanno partecipato all'esame*? Oppure gli studenti *che non l'hanno mai superato?* Come vediamo, la situazione diventa diversa:

- Nel primo esempio, se non vedo uno studente fino alla penultima riga non posso dare per scontato che non si sia presentato a quell'esame, dato che potrebbe essere presente come ultima tupla della relazione.
- Nel secondo esempio, invece, se leggo che lo studente con codice 003 non ha superato l'esame in una tupla, non posso decretare direttamente che non lo abbia mai superato e dunque inserirlo nella relazione di destinazione, in quanto potrebbe esserci una tupla successiva in cui viene invece detto che lo studente ha superato tale esame.

Per questo tipo di interrogazioni, dobbiamo cambiare il modo di ragionare: non più con il quantificatore $\exists$, bensì con il quantificatore $\forall$ oppure $\nexists$.

Ragionare secondo quantificatori globali, che dunque prendono in esempio tanti elementi insieme, è sempre più difficile. Potremmo dunque ragionare al contrario, **negando** il quantificatore globale per ottenerne uno unario.

Definizione 6.10.1: Negazione di $\forall$

Data una proposizione del tipo "$\forall$ oggetti VALE condizione", la sua **negazione** sarà la proposizione "$\exists$ **oggetto per cui NON VALE CONDIZIONE**".

Facciamo un esempio pratico: la negazione della frase "Tutti i miei amici si chiamano Francesco" è "Ho ALMENO UN amico che non si chiama Francesco". Come vediamo, siamo riusciti a trasformare un quantificatore globale in uno unario semplicemente negando la proposizione.

Per testarne il funzionamento, proviamolo su una richiesta vera.

Esempio 6.10.1: Interrogaione con negazione di quantificatori globali

Supponiamo la richiesta sia "*Trovare tutti gli studenti che hanno superato tutti gli esami a cui hanno partecipato*".
Se fossimo rimasti con la mentalità precedente, sarebbe stato un disastro: per ogni studente avremmo dovuto memorizzare se ha passato un esame e solo se tutti fossero stati veri avremmo potuto metterlo nella nuova relazione.

Adesso sappiamo però che la negazione ci è amica, e dunque la richiesta risulta esser equivalente a "*Studenti che NON SONO MAI STATI BOCCIATI*". E qui la soluzione diventa banale: prendiamo la relazione ottenuta dalla Theta Join tra `Studenti` ed `Esami` e verifichiamo l'esito dell'esame per ogni tupla: se leggiamo `Esito = "Bocciato"`, allora questa tupla **non deve stare nella relazione richiesta**. Tuttavia, dato che stiamo ragionando al **complemento**, dovremo memorizzare questa tupla in una **relazione contenente solo gli studenti che non rispettano la condizione di partenza**.
A quel punto, scansionate tutte le tuple, prendiamo la **relazione studenti di partenza e applichiamo la differenza con la relazione degli studenti bocciati almeno una volta**: il risultato di tale operazione insiemistica sarà anche la risposta alla nostra query.

In altri casi, a seconda di come è stata costruita la base di dati, potremmo ritrovarci che le informazioni per una query siano tutte nella stessa tabella, però appartenenti a tuple diverse che devono essere collegate l'un l'altra. Di seguito si presenta un esempio.

Esempio 6.10.2: Interrogazione con Auto-Join

Supponiamo di avere una relazione `Impiegato` con il seguente schema `(Codice (PK), Cognome, Stipendio, CodCapo (FK))` e di voler ottenere *tutti gli impiegati che hanno uno stipendio maggiore di quello del loro capo*. All'interno della relazione `Impiegato` sono infatti memorizzati anche i capi.

Il fatto che le informazioni siano tutte nella stessa relazione non deve spaventarci, in quanto possiamo ragionare in maniera quasi completamente uguale a quando usavamo due relazioni: facciamo la Join tra le due relazioni ed estraiamo le informazioni.
L'unico cambiamento nei casi delle Auto-Join risiede nel fatto che, essendo la stessa tabella, avremo i campi *tutti con lo stesso nome*, e dunque molto probabilmente si verificheranno problemi di correttezza dei dati. La soluzione è quindi rinominare una delle due tabelle e i suoi campi.

1. Creiamo una copia ridenominata per la tabella contenente tutti i capi.

$$\text{Capi} = \rho_{\text{CodC, CognC, StipC, CC}\leftarrow\text{Codice, Cognome, Stipendio, CodCapo}}(\text{Impiegato})$$

2. Eseguiamo il Theta-Join tra la relazione `Impiegati` e la nuova relazione `Capi`.

$$R_1 = \text{Impiegati} \bowtie_{\text{CodCapo=CodC} \wedge \text{Stipendio>StipC}} \text{Capi}$$

3. Solo per un output più carino, eseguiamo una proiezione per estrarre solo codice e cognome di tale impiegato

$$R_{\text{finale}} = \pi_{\text{Codice, Cognome}}(\text{R}_1)$$

Esempio 6.10.3: Query complessa con Auto-Join

Se invece, sempre in riferimento allo schema di relazione precedente, volessimo ottenere *codice e cognome dei capi che guadagnano più di tutti i loro dipendenti*, dovremmo usare la struttura a doppia negazione.

1. Otteniamo la tabella di tutti i capi. Supponiamo di avere la relazione `Capi` definita come prima, e anche la relazione R_1 definita come Theta-Join tra impiegati e i loro capi. Per estrarre adesso solo i capi, cioè nome e cognome di ogni capo, ci basta fare una proiezione su questa relazione:

$$\text{TuttiCapi} = \pi_{\text{CodC, CognC}}(\text{R}_1)$$

   Ci servirà per l'ultimo passaggio.

2. Adesso dobbiamo ottenere tutti i capi che guadagnano più di *tutti* i propri dipendenti. Questo si ottiene negando la proposizione, e dunque equivale a trovare **almeno**

un capo che guadagna meno di un suo dipendente.

$$\text{CapiScartati} = \text{Impiegato} \bowtie_{\text{CodCapo=CodC} \wedge \text{Stipendio>=StipC}} (\text{Capo})$$

3. Adesso, dall'insieme di tutti i capi che abbiamo trovato nel primo passaggio, possiamo rimuovere quelli che non rispettano la richiesta iniziale, semplicemente con l'operatore di differenza.

$$R_{\text{finale}} = \text{Capi} - \text{CapiScartati}$$

7

Il linguaggio SQL

Al fine di creare i database, le tabelle, stabilire i vincoli tra queste ed effettuare operazioni di interrogazione sui dati era necessario un apposito linguaggio con cui fare tutto questo.
Nasce così **SQL (Structured Query Language)**, il linguaggio diventato ormai di riferimento per tutti i DBMS moderni.

Definizione 7.0.1: Sottolinguaggi di SQL

Il linguaggio SQL è un grande contenitore di moltissime istruzioni di vario genere. Nonostante la suddivisione logica di queste istruzioni non sia importante ai fini del calcolo, si tratta comunque di una nozione utile ai fini della progettazione o dell'uso di un DBMS.
Le varie istruzioni del linguaggio SQL si suddividono in 4 sottolinguaggi:

- **DDL (Data Definition Language)**: contiene tutte le istruzioni che permettono la creazione/modifica/eliminazione di database e tabelle (es: `CREATE TABLE (...)`, `CREATE DATABASE ..., ...`).
- **DML (Data Manipulation Language)**: comprende le istruzioni utili alla manipolazione dei dati all'interno delle tabelle, permettendo ad esempio istruzioni di inserimento, modifica o cancellazione di un dato (es: `INSERT INTO ... VALUES (...)`).
- **DQL (Data Query Language)**: si tratta del linguaggio che comprende tutte le istruzioni utili all'interrogazione dei dati (es: `SELECT ... FROM ... WHERE ...`).
- **DCL (Data Control Language)**: gestisce i permessi utente e limita l'accesso ai dati (`GRANT, REVOKE, ...`).

7.1 Il linguaggio di definizione dei dati: DDL

Nella seguente sezione andremo ad analizzare tutte le operazioni fondamentali che riguardano il linguaggio DDL per la creazione, modifica, cancellazione di tabelle, definizioni di dati e domini, creazione di vincoli intra e inter-relazionali.

7.1.1 Domini di dati

Come abbiamo già discusso nella formalizzazione del modello relazionale per la memorizzazione dei dati, ad ogni attributo è associato un dominio di valori che possono essere assunti da esso. Nel linguaggio SQL ci vengono forniti dei domini di dati predefiniti, ma ci è anche dato modo di crearne di nostri.
Di seguito una tabella in cui riassumiamo i domini principali forniti dal linguaggio SQL. Chiaramente ne esistono molto di più, ma questi sono gli essenziali nella progettazione di una base di dati standard.

Nome in SQL	Descrizione
`CHAR(size)`	Rappresenza una stringa di lunghezza fissa di `size` caratteri. Il massimo di caratteri rappresentabili è 255.
`VARCHAR(size)`	Rappresenta una stringa a lunghezza variabile il cui numero massimo di caratteri è 65535.
`INT`	Rappresentazione standard di un valore intero. Ci sono anche altri tipi di dato per cui varia il range di rappresentazione dei valori.
`FLOAT`	Memorizza valori decimali con una "bassa precisione".
`DOUBLE`	Il tipo di dato più preciso per memorizzare valori decimali.
`BLOB/CLOB`	Predisposti a contenere grandi immagini e testi, ma non possono essere oggetto di interrogazioni.
`BOOLEAN/BOOL`	Modo standard per memorizzare valori come TRUE (tutti i numeri diversi da 0) o FALSE (unicamente 0).

`DATE`	Memorizza una data nel formato "YYYY-MM-DD".
`TIME`	Memorizza l'ora nel formato "hh:mm:ss".
`DATETIME`	Una combinazione di `DATE` e `TIME` che si presenta nel formato "YYYY-MM-DD hh:mm:ss".

Per creare invece dei domini personalizzati, si usa l'istruzione `CREATE DOMAIN`. Per quanto la sintassi ora come ora possa non essere falmiliare, viene lasciato di seguito un esempio:

```
CREATE DOMAIN AnnoAssunzione AS INT DEFAULT NULL CHECK (value >= 1990)
```

Provando a spiegare la precedente istruzioni, stiamo

1. Usando le istruzioni `CREATE DOMAIN` per "annunciare" di voler creare un dominio personalizzato.
2. Definendo il nome del dominio (AnnoAssunzione).
3. Keyword `AS` per indicare che tutto ciò che segue saranno le istruzioni per la definizione del dominio (tutto ciò che ci risparmieremo di scrivere ogni volta).
4. Definiamo il tipo di dominio a cui applicare le restrizioni. In questo caso `INT`.
5. Stabiliamo un valore di default, cioè in caso il dato non sia ancora stato inserito nella tabella. In questo caso abbiamo impostato `DEFAULT NULL`.
6. Definiamo l'effettiva *restrizione* da applicare al tipo di dato di partenza.
7. `value` è un attributo specifico di ogni dominio di dato e rappresenta l'effettivo valore che si è provato ad inserire nella tabella. In questo caso vogliamo imporre che tutti i dati che apparterranno al dominio "AnnoAssunzione" debbano avere un valore >= 1990.

7.1.2 Creazione di una tabella e constraint

Di seguito vediamo come definire una tabella in SQL (mediante istruzioni del linguaggio DDL) e analizzeremo tutti i *constraint* che possiamo imporre sullo schema di questa.
Innanzitutto, partiamo dalle basi: la creazione di una tabella avviene con l'istruzione `CREATE TABLE nomeTabella (...);` Mostriamo di seguito un esempio di creazione di due tabelle e della creazione dei vincoli inter-relazionali e intra-relazionali.

```
CREATE TABLE Dipartimento(
Codice CHAR(15) PRIMARY KEY,
Nome CHAR(30) UNIQUE NOT NULL
);

CREATE TABLE Impiegato(
Matricola CHAR(6) PRIMARY KEY,
CF CHAR(16) UNIQUE NOT NULL,
Cognome CHAR(30) NOT NULL,
Stipendio DOUBLE DEFAULT 0,
FK_Dipartimento CHAR(15) REFERENCES Dipartimento(Codice)
);
```

Analizziamo adesso ogni istruzione, spieghiamola e forniamo eventuali varianti di sintassi.

- In generale, la definizione di un attributo segue la sintassi `NOME_ATTRIBUTO DOMINIO CONSTRAINT`.
- Se vogliamo indicare che un attributo è chiave primaria di una relazione (ossia l'attributo che identifica univocamente ogni tupla della relazione) possiamo procedere in due modi:
  1. Come nell'esempio, scrivendo `PRIMARY KEY` come constraint dell'attributo.
  2. Prima della fine della definizione della tabella, possiamo scrivere `PRIMARY KEY (Attributo1, Attributo2, ...)` per definire una chiave primaria composta da uno o più attributi. Questa modalità è **obbligatoria** quando abbiamo relazioni in cui la chiave primaria è composta da più di un attributo, in quanto scrivere `PRIMARY KEY` come constraint su due attributi diversi risulterebbe in un errore.
- Per definire un vincolo di chiave esterna tra due relazioni possiamo anche qui procedere in due modi:
  1. Come nell'esempio, scrivendo `NOME_ATTRIBUTO TIPO REFERENCES TABELLA_DI_PROVENIENZA(NOME_ATTRIBUTO_DI_PROVENIENZA)`
  2. Utilizzando una sintassi simile a quella per le chiavi primarie composte, ovvero `FOREIGN KEY(NOME_ATTRIBUTO) REFERENCES TABELLA_DI_PROVENIENZA(NOME_ATTRIBUTO_DI_PROVENIENZA)`.
     Di nuovo, questa modalità è **obbligatoria** quando la chiave esterna ereditata dalla relazione sia una chiave primaria composta nella relazione di partenza.

7.1.3 Creazione dei vincoli inter-relazionali

Il constraint `NOT NULL` indica che **non sono ammessi valori nulli per quell'attribito**.

Il constraint `UNIQUE` indica che ogni **ogni tupla deve avere un valore diverso per quell'attributo**.

Il constraint `CHECK` indica un **controllo sul valore di uno o più attributi**. Viene effettuato il controllo che il valore inserito all'interno della tupla rispetti la condizione indicata tra parentesi tonde, altrimenti l'inserimento non va a buon fine. Se un `CHECK` coinvolge due o più attributi, deve necessariamente essere definito come vincolo della tabella (dunque non su una riga di un attributo).
Un esempio classico può essere impedire che la data di fine risulti inferiore della data di inizio. Il constraint da impostare è dunque `CHECK (dataFine >= dataInizio)`.

Proposizione 7.1.1: Definizione dei constraint

Come abbiamo visto, possiamo definire i constraint in due modi:

- Inline, dunque sulla stessa riga su cui vogliamo far valere il constraint (metodo utilizzabile solo se il suddetto constraint coinvolge un solo attributo).
- In una riga autonoma della tabella, e qui possiamo coinvolgere quanti attributi vogliamo.

Se volessimo **dare un nome al constraint**, specifico per la tabella in cui ci troviamo, possiamo usare la parola chiave `CONSTRAINT` e specificando di seguito il nome che vogliamo dare al vincolo. Questo ci sarà utile nel futuro se mai volessimo eliminare o modificare un vincolo.
Ad esempio:

`Matricola INT CONSTRAINT Chiave_Primaria PRIMARY_KEY` (inline)
`CONSTRAINT Chiave_Primaria PRIMARY_KEY(Matricola)` (non inline)

Se invece volessimo aggiungere un constraint dopo aver già creato la tabella:

```
ALTER TABLE Studente ADD CONSTRAINT CHK_Anno_Nascita CHECK(DataNascita >
1920)
```

e per rimuoverlo:

```
ALTER TABLE Studente DROP CONSTRAINT CHK_Anno_Nascita
```

Come vediamo, per rimuovere un constraint è *essenziale che questo abbia un nome identificativo.*

Osservazione 7.1.1: Il constraint PRIMARY KEY

Il constraint `PRIMARY KEY` include implicitamente anche i vincoli `UNIQUE` (cioè che ogni valore memorizzato per quell'attributo deve essere univoco e non devono apparire valori duplicati) e `NOT NULL` (ovvero che non sono ammessi valori nulli per quel campo), oltre all'indicizzazione da parte del DBMS (ma qui si scende al modello fisico).

Osservazione 7.1.2: Importanza dei domini nelle `FOREIGN KEY`

Come visto nell'esempio, affinché la creazione di un vincolo di integrità inter-relazionale tra due tabelle vada a buon fine è importante che l'attributo/gli attributi che si vogliono mettere in relazione abbiano **lo stesso dominio**. E ciò non vuol dire semplicemente che siano entrambi due `CHAR`, ma vuol dire ad esempio che se è stata configurata una dimensione per la stringa dei caratteri, questa debba essere rispettata (es: se nella tabella di partenza avevo `CHAR(16)`, anche in quella di arrivo deve essere `CHAR(16)`).

Per notazione, la tabella da cui *proviene* la chiave esterna (dunque la tabella in cui gli attributi sono chiave primaria) è detta **master**. La tabella in cui vengono importati tali valori viene invece detta **slave**.

Nella creazione di una tabella, si possono anche definire i comportamenti da attuare quando un certo valore viene cancellato o aggiornato. Questo è particolarmente importante quando si lavora con chiavi esterne, in quanto è importante definire come si devono comportare tutte le tabelle che implementano la chiave esterna quando la chiave interna di provenienza viene eliminata oppure gli viene cambiato valore.

Esempio 7.1.1: Opzioni di aggiornamento chiave esterna

Supponiamo di avere la definizione delle tabelle viste nell'esempio precedente:

```
1       CREATE TABLE Dipartimento(
2       Codice CHAR(15) PRIMARY KEY,
3       Nome CHAR(30) UNIQUE NOT NULL
4       );
5
6       CREATE TABLE Impiegato(
7       Matricola CHAR(6) PRIMARY KEY,
8       CF CHAR(16) UNIQUE NOT NULL,
9       Cognome CHAR(30) NOT NULL,
10      Stipendio DOUBLE DEFAULT 0,
```

```
11      FK_Dipartimento CHAR(15) REFERENCES Dipartimento(
            Codice)
12      );
```

Se ad esempio il codice del dipartimento cambiasse, oppure un dipartimento con un certo codice venisse eliminato, dovremmo indirizzare il DBMS su come comportarsi.

Ci sono due modalità principali:

- `ON DELETE`, per definire il comportamento da attuare quando la **chiave primaria di riferimento viene cancellata**.
- `ON UPDATE`, per definire il comportamento da attuare quando la **chiave primaria di riferimento viene aggiornata**.

Per ciascuna di queste modalità abbiamo due opzioni:

1. `CASCADE`, significa che **propaghiamo quel comportamento** (cancellazione oppure aggiornamento) **su tutti i record che contenevano quel valore**. Usare `ON DELETE CASCADE` è molto pericoloso, in quanto **cancella interamente i record che avevano come valore della chiave esterna il valore della chiave primaria appena cancellata**.
2. `SET NULL`, significa che **ogni campo che conteneva il valore che abbiamo cancellato/modificato viene impostato a NULL**.

Il linguaggio SQL ci mette anche a disposizione gli strumenti per creare degli **indici**. Il concetto di indice verrà più approfonditamente trattato in seguito, ma per ora basti sapere che la maggior parte dei DBMS utilizza l'**indicizzazione dei campi per poterli accedere più velocemente durante le operazioni di interrogazione** e per identificare univocamente quell'attributo nella base di dati. Come ci si può immaginare, quando creiamo una **chiave primaria** viene **automaticamente definito un indice su quel campo**: ecco perché è importante scegliere come PK un insieme di attributi anche essenziali nella maggior parte delle operazioni di interrogazione.

La definizione di un indice permette di:

1. **Accedere in modo quasi istantaneo ai record contenenti l'attributo** che abbiamo indicizzato. Senza quest'operazione, il database dovrebbe "manualmente" scorrere milioni di righe per trovare quella corretta (operazione `WHERE` nelle interrogazioni).
2. **Migliorare le prestazioni quando si coinvolgono più tabelle** (operazione `JOIN` nelle interrogazioni).

3. **Rendere più rapide operazioni di ordinamento e raggruppamento delle tuple** secondo una specifica condizione (operazioni `ORDER BY` e `GROUP BY` delle interrogazioni).

Possiamo definire un indice personalizzato semplicemente con:

```
CREATE INDEX nome_indice ON tabella(attributo1, attributo2, ...)
```

Tuttavia non bisogna abusare degli indici in quanto **rallentano le operazioni di modifica della base di dati** e **consumano spazio aggiuntivo sul disco**.

7.1.4 Modifica degli schemi

Il linguaggio DDL serve anche a modificare le tabelle per aggiungere/rimuovere attributi e vincoli.

- **Aggiungere una colonna ad una tabella**: `ALTER TABLE tabella ADD COLUMN colonna TIPO CONSTRAINT`. Se la tabella contiene già dei dati, questa nuova colonna viene popolata con valori nulli.
- **Rimuovere una colonna da una tabella**: `ALTER TABLE tabella DROP COLUMN colonna`.
- **Modificare il tipo di una colonna**: `ALTER TABLE tabella MODIFY colonna TIPO`.
- **Rinominare una colonna**: `ALTER TABLE tabella RENAME COLUMN vecchio_nome_colonna TO nuovo_nome`
- **Rinominare una tabella**: `ALTER TABLE tabella RENAME TO nuovo_nome_tabella`
- **Cancellare una tabella**: `DROP TABLE tabella`. Cancella lo schema e tutti i dati al suo interno.

7.1.5 Creazione delle viste

In riferimento al modello ANSI/SPARC visto nella Sezione 2.1, ricordiamo di come, in base ai propri permessi nella base di dati, dovrebbe essere possibile accedere solo ad una porzione dei dati memorizzati.
Questa tecnica si ottiene mediante la creazione delle cosiddette **viste**, ossia delle **tabelle virtuali create a partire da una query** formulata usando il linguaggio QL.
La sintassi di base per la creazione di una vista è:

```
CREATE VIEW nome_vista AS
<descrizione di una query>
```

Mediante una vista, potremmo fare in modo che gli utenti accedano solo ai dati della vista, dunque già di per sé una porzione filtrata dei dati risultato di una speciale interrogazione.

7.1.6 Basi di dati attive e Trigger

I DBMS tradizionali sono passivi, cioè tutte le operazioni vengono eseguite a richiesta dell'utente. Ci potrebbero essere dai casi in cui è però necessario effettuare dei controlli *asincroni* sulle operazioni compiute, ovvero controlli su eventi scatenabili potenzialmente in qualsiasi momento. Serve dunque un DBMS che sia in grado di rispondere a questa categoria di eventi: parliamo qui di **DBMS attivi (ADBMS)**.
In realtà, abbiamo già visto un esempio di controllo attivo effettuato dal DBMS quando parlavamo dei vincoli di integrità referenziale: per quanto il concetto di *chiave esterna* sia intuitivo e definito a livello anche matematico, nel pratico il DBMS deve controllare che, inserendo un record in una tabella *slave*, il valore assunto dall'attributo definito come chiave esterna sia presente nell'attributo di riferimento della tabella *master*. Questo tipo di controllo può essere effettuato solo "*a runtime*", dunque una volta che si prova effettivamente ad inserire o modificare questi dati, e non è un'eccezione che possiamo in qualche modo prevedere. Non possiamo prevederla, ma possiamo gestirla.
Questo è solo uno dei tanti esempi di eventi asincroni gestibili dai DBMS, che a tal proposito ci mettono uno strumento potentissimo: i **trigger**. Il paradigma adottato dai trigger si dice di tipo **ECA - Evento-Condizione-Azione**, e può essere interpretato nel seguente modo:

1. Allo scatenarsi di un **evento**...
2. Se si verifica una certa **condizione**...
3. Si deve procedere con un'opportuna **azione**.

Questa natura asincrona della gestione degli eventi ricorda per molti versi il pattern Observer della programmazione classica, quindi per chi è familiare in merito potrebbe essere più semplice da comprendere.
Di seguito vediamo la sintassi classica per la definizione dei trigger: si premette che non è una sintassi leggera, e che dunque verrà opportunamente analizzata sotto.

```
1  CREATE TRIGGER NomeTrigger
2  { BEFORE | AFTER } Evento ON TabellaTarget
3  [ REFERENCING { OLD ROW AS NomeVecchiaRiga | NEW ROW AS
      NomeNuovaRiga } | { OLD TABLE AS NomeVecchiaTabella | NEW
       TABLE AS NomeNuovaTabella } ]
4  [ FOR EACH { ROW | STATEMENT } ]
5  [ WHEN Condizione ]
6  BEGIN
7    Comandi SQL da eseguire
8  END;
```

Allora, vediamo passo passo ogni istruzione:

- `CREATE TRIGGER NomeTrigger`: si definisce un nuovo trigger e ne si specifica il nome.
- Nella seconda riga, si può innanzitutto decidere **quando** il trigger deve agire (le due opzioni sono, ovviamente, mutualmente esclusive):
  - `BEFORE`: il trigger intercetta l'operazione **prima** che venga effettivamente scritta su disco. E' più comune, in quanto serve per effettuare controlli a priori su inserimenti o cambiamenti di dati.
  - `AFTER`: il trigger si attiva solo dopo che l'operazione è stata committata su disco. Serve generalmente per mantenere log di dati.
- Dobbiamo anche specificare l'evento per il quale si deve attivare questo trigger. I tipi di evento sono chiaramente inserimento (`INSERT`), modifica (`UPDATE`) e cancellazione (`DELETE`).
- Specifichiamo con `ON Tabella` anche la tabella sul quale deve avvenire il preciso evento per far sì che si scateni il trigger.
- La clausola `REFERENCING` e tutto ciò che ne segue è completamente opzionale, ma si tratta di un meccanismo per accedere ai dati che stanno subendo la modifica. Infatti in questa clausola si può decidere di attribuire un nome di variabile alla vecchia/nuova (`OLD/NEW`) riga/tabella (`ROW/TABLE`), rinominandola grazie alla clausola `AS`.
- La riga `FOR EACH ROW | STATEMENT` definisce *quante volte dovrà essere eseguito il trigger*:
  - Specificando `ROW`, stiamo dicendo che il trigger deve essere eseguito **una volta per ogni singola riga modificata dall'evento**.
  - Con `STATEMENT` si fa in modo che il trigger venga eseguito **una singola volta per comando**. E con *comando* si intende una **singola** `INSERT`, `UPDATE` o `DELETE`.
- `WHEN Condizione` serve a filtrare ulteriormente l'esecuzione del trigger, che eseguirà i seguenti comandi SQL solo se la condizione risulterà vera.

Alla complessità di questo comando si contrappone la semplicità di eliminazione di un trigger: `DROP TRIGGER NomeTrigger`.

Osservazione 7.1.3: Evento `UPDATE`

Un'importante osservazione va fatta riguardo agli eventi di tipo `UPDATE`. Potremmo voler effettuare un controllo solo su *un numero limitato di attributi che vengono modificati*, non su tutta la riga. In tal caso scriveremo:

```
...
{ BEFORE | AFTER } UPDATE OF ListaAttributi ON
   Tabella
...
```

La cosa importante da sapere è che, se specifichiamo un attributo$_i$ all'interno di `ListaAttributi`, il trigger si attiverà **ogni volta che tale attributo sarà coinvolto in un'operazione di `UPDATE`**, indipendentemente dal fatto che sia solo o sia con altri attributi.

Osservazione 7.1.4: La clausola `REFERENCING`

Quando viene scatenato un evento asincrono a cui un trigger deve reagire, viene memorizzato lo stato corrente della tabella `Target`. Inoltre, viene anche creata temporaneamente una nuova tabella contenente lo stato atteso **dopo** aver effettuato l'operazione di modifica specificata dall'evento. Queste due tabelle vengono dette **transition table**, e sono proprio quelle che ci permettono di riferirci alle vecchi/nuove righe/tabelle nella clausola `REFERENCING`.

Osservazione 7.1.5: Trigger e vincoli di integrità

Supponiamo di aver definito, nel seguente ordine, i seguenti trigger e vincoli:

```
CREATE TRIGGER Trigger1 AFTER UPDATE ON Tabella1 ...
CREATE TRIGGER Trigger2 BEFORE UPDATE ON Tabella1 ...
CREATE TRIGGER Trigger3 AFTER UPDATE ON Tabella1 ...
ALTER TABLE Tabella1 ADD CONSTRAINT Vincolo1 ...
```

Supponiamo adesso di voler effettuare un'operazione di modifica (`UPDATE`) sulla tabella `Tabella1`. L'ordine in cui questi controlli verranno effettuati sarà il seguente:

1. `Trigger2`: avendolo definito come `BEFORE UPDATE`, questo trigger viene eseguito *prima* della modifica effettiva della tabella e del commit su disco.
2. Viene eseguita l'effettiva modifica dei dati su `Tabella1`.
3. `Vincolo1`: avendo completato la modifica, è necessario controllare che non siano stati violati vincoli di dominio o di integrità referenziale.
4. `Trigger1`: essendo stato definito come `AFTER UPDATE`, una volta che il controllo del vincolo (si suppone) è andato a buon fine, si può procedere con il trigger da

eseguire *dopo* aver modificato i dati.

5. `Trigger3`: anche questo è stato definito come `AFTER UPDATE`, ma viene eseguito *dopo* `Trigger1` in quanto definito dopo.

Dunque, in generale, l'ordine da seguire è:

$$\texttt{BEFORE} \rightarrow \texttt{Modifica} \rightarrow \texttt{Vincoli} \rightarrow \texttt{AFTER}$$

e, all'interno di ogni categoria, si procede per ordine di definizione.

Per consolidare definitivamente il discorso sui trigger, per il quale però si invita il lettore ad esercitarsi anche in solitaria, si lascia un esempio.

Esempio 7.1.2: Esempio di uso dei trigger

Supponiamo che un'azienda voglia impedire qualsiasi tipo di operazione sulla base di dati prima e dopo una certa ora (supponiamo prima dell'apertura, alle 9, e dopo la chiusura, alle 18). Il trigger da scrivere dovrà essere il seguente.

```
CREATE TRIGGER BloccaModificheNotturne
BEFORE UPDATE ON Tabella
FOR EACH STATEMENT
BEGIN
  IF CURRENT_TIME > '18:00:00' OR CURRENT_TIME < '
     09:00:00' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Operazione non consentita
       fuori dall orario d ufficio!';
  END IF;
END;
```

Listing 7.1: Per prevenire operazioni su ogni tabella, si dovrà creare un trigger del genere per ogni tabella della base di dati.

Come vediamo, in questo caso stiamo usando `FOR EACH STATEMENT` come granularità di esecuzione in quanto non ci interessa sapere quante/quali righe verranno modificate, bensì solo il fatto che vengano modificate (dunque l'esecuzione dell'operazione) è di per sè un problema.

Vediamo un altro esempio: supponiamo di voler prevenire che ad un utente venga ridotto lo stipendio.

```
CREATE TRIGGER PrevieniRiduzioneStipendio
BEFORE UPDATE ON Dipendente
REFERENCING OLD ROW AS VecchiaRiga NEW ROW AS
   NuovaRiga
FOR EACH ROW
WHEN (NuovaRiga.Stipendio < VecchiaRiga.Stipendio)
BEGIN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Impossibile ridurre lo
     stipendio!';
END;
```

Stavolta invece operiamo per riga, in quanto l'operazione di `UPDATE` potrebbe modificare più di una riga e in ogni caso abbiamo necessità di fare riferimento alla riga modificata.

Osservazione 7.1.6: Vincoli interpretati come trigger

Come detto anche all'inizio del paragrafo riguardante i trigger, alcuni esempi di controlli asincroni possono essere individuati anche sui vincoli di dominio o di chiave esterna.
Non verrà fornita l'implementazione in "versione trigger" di tali vincoli, ma si invita il lettore a pensare a come questi potrebbero essere scritti.

7.2 Il linguaggio di interrogazione dei dati: DQL

Lo studio degli operatori dell'algebra relazionale è fondamentale per capire la logica che avviene dietro alle interrogazioni di una base di dati che segue il modello relazionale. Avendo definito il linguaggio formale e matematico da cui prendere le fondamenta, possiamo spostarci sull'effettivo **linguaggio che useremo per ottenere le informazioni sulla nostra base di dati**: il **Data Query Language (DQL)**.

Osservazione 7.2.1: Natura del linguaggio DQL

Come abbiamo visto, l'algebra relazionale è un linguaggio *procedurale*: occorre specificare l'esatta sequenza delle operazioni da effettuare per ottenere l'informazione richiesta.
Il linguaggio DQL è invece un linguaggio **dichiarativo**: ci basta dire *cosa* trovare, *dove* trovarlo e *quali controlli effettuare sui dati*.

7.2.1 I comandi essenziali

Sono 3 i comandi essenziali per la scrittura di una query nel linguaggio DQL.

La clausola `SELECT`: la primissima e più basilare operazione che può essere fatta con il linguaggio DQL è **indicare quali dati vogliamo visualizzare come risultato della nostra interrogazione**. L'operazione di `SELECT` è infatti il diretto corrispettivo della **proiezione** nell'algebra relazionale.

La clausola `FROM`: al fine di recuperare i dati da una o più tabelle, dobbiamo però comunicare al linguaggio *da quali tabelle vogliamo prendere questi dati*. Ed è proprio qui che entra in gioco la clausola `FROM`, in seguito alla quale dovremo indicare i nomi delle tabelle da esaminare.

La clausola `WHERE`: si tratta di una clausola opzionale, che permette di filtrare le tuple esaminate mediante una condizione. Come si può immaginare dalla descrizione appena fornita, quest'operatore è il corrispettivo della **selecione** nell'algebra relazionale.

```
1  SELECT attributo1, attributo2, ...
2  FROM tabella1, tabella2, ...
3  [ WHERE condizione ]
```

Listing 7.2: Sintassi di base di una query nel linguaggio DQL

Forniamo di seguito la tabella `Persone` che prenderemo come riferimento per vari esempi di interrogazioni nel corso di questa sezione.

Nome	Eta	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filippo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41

Sergio	85	35
Luisa	75	87

Esempio 7.2.1: Primo esempio di query

Data la precedente tabella, supponiamo di voler estrarre da questa "nome e reddito di tutte le persone".
Seguendo la logica dichiarativa del linguaggio DQL, prima di impostare la sintassi dobbiamo porci le seguenti domande:

- **Quali dati** voglio mandare a schermo? Nel nostro caso sono il *nome* e il *reddito*.
- **Da quali tabelle** prendiamo questi dati? Nel nostro caso, esclusivamente dalla tabella *persone*.
- **Quale condizione devono rispettare le tuple** in cui sono contenuti certi dati? Nel nostro caso, vogliamo sapere il nome e reddito di *tutte* le persone, dunque non poniamo condizioni.

Alla luce di queste considerazioni, l'effettiva query da scrivere sarà

```
1  SELECT Nome, Reddito
2  FROM Persone
```

Il risultato di questa interrogazione sarà a sua volta una tabella (come nell'algebra relazionale) e nello specifico sarà la tabella `Persone` senza però i valori dell'attributo *Eta*.

Esempio 7.2.2: Esempio di query con condizione

Se invece volessimo ottenere i dati solo ed unicamente delle persone di età inferiore ai 30 anni, dovremo stavolta applicare una condizione secondo cui le tuple verranno filtrate. Supponendo di voler visualizzare nome, reddito ed età di queste persone, la query da scrivere sarà la seguente.

```
1  SELECT Nome, Eta, Reddito
2  FROM Persone
3  Where Eta < 30
```

Come funziona questo meccanismo di filtraggio? Banalmente, la condizione viene valutata tupla per tupla: si prendono prima tutte le tabelle coinvolte, poi tupla per tupla si valuta la condizione scritta nella `WHERE` e si decide se scartare o meno la tupla dal risultato finale.

Nome	Eta	Reddito	(Filtrata?)
Andrea	27	21	eta $<$ 30 ? sì
Aldo	25	15	eta $<$ 30 ? sì
Maria	55	42	eta $<$ 30 ? no
Anna	50	35	eta $<$ 30 ? no
Filippo	26	30	eta $<$ 30 ? sì
Luigi	50	40	eta $<$ 30 ? no
Franco	60	20	eta $<$ 30 ? no
Olga	30	41	eta $<$ 30 ? no
Sergio	85	35	eta $<$ 30 ? no
Luisa	75	87	eta $<$ 30 ? no

Nella tabella risultante (che non verrà proposta per brevità) figureranno solo le tuple che hanno "superato" il filtraggio.

Osservazione 7.2.2: Abbreviazioni del DQL

E' doveroso citare anche alcune abbreviazioni e scorciatoie visive che possono essere utilizzate durante la stesura di una query.

Rinominare una tabella: è una tecnica che si usa quando si hanno nomi di tabelle molto lunghi e ci si vuole riferire a loro più brevemente. Ovviamente, **questa è un'operazione solo logica e non va a cambiare fisicamente il nome della tabella** (non è una `ALTER TABLE` insomma). Per ottenere questa ridenominazione è sufficiente scrivere `FROM nomeOriginale AS ridenominazioneLogica`. Così facendo, nel corso della query ci si dovrà riferire alla tabella col nome logico e non con quello originale.

Rinominare un attributo: è una tecnica che si usa generalmente quando si hanno nomi di attributi poco descrittivi dell'attributo stesso. Ovviamente, **questa è un'operazione solo logica e non va a cambiare fisicamente il nome della**

tabella (non è una `ALTER TABLE` insomma). Per ottenere questa ridenominazione è sufficiente scrivere `SELECT Nome AS NomePersona`. Così facendo, nel corso della query ci si dovrà riferire all'attributo col nome logico e non con quello originale.

Riferirsi a specifici attributi: questa tecnica è utile nel caso si abbiano due tabelle con lo stesso nome di uno o più attributi, e dunque scrivere semplicemente `Nome`, ad esempio, risulterebbe in un conflitto di interessi. La soluzione in questo caso è di anteporre al nome dell'attributo quello della tabella di provenienza, utilizzando la "dot-notation" vista anche nelle classi del linguaggio Java. Prendendo in esempio la tabella precedente, se volessimo fare riferimento al nome potremmo scrivere `persona.Nome`.

Espressioni matematiche nelle query

Possiamo anche effettuare operazioni matematiche sui dati con i classici operatori `+, -, *, /, %` e l'aggiunta di un operatore `POW(x, y)` che ci permette di elevare alla potenza di `y` il numero `x`. Le regole seguite dagli operatori, come ci si può immaginare, sono quelle dell'algebra di base che conosciamo dalle elementari.

```
1  SELECT Reddito/2 AS RedditoSemestrale
2  FROM Persone
3  WHERE Nome = "Luigi";
```

Listing 7.3: Esempio di query con operazioni matematiche nella `SELECT`

Chiaramente tali operatori non sono limitati alla clausola `SELECT`, bensì possono essere utilizzati anche per controlli nella clausola `WHERE`.

Il comando LIKE

Abbiamo poi ad esempio anche l'operatore `LIKE` che ci permette di controllare se una stringa di caratteri rispetta una certa sintassi: se ad esempio scrivessimo `WHERE Nome LIKE "A_d%"` staremo chiedendo tutte le tuple in cui il nome ha 'A' come prima lettera, una qualsiasi lettera come seconda, 'd' come terza lettera e poi una qualsiasi sequenza di lettere di qualsiasi lunghezza (rappresentata dal %).

Controllo dei valori NULLI

Per controllare che un valore sia o meno `NULL` non si usano gli operatori `=` o `<>/!=` (diverso), bensì degli speciali comandi `IS NULL` e `IS NOT NULL`.

L'operatore `DISTINCT`

Se volessimo replicare esattamente il comportamento dell'operatore di proiezione nell'algebra relazionale, dovremmo usare l'operatore `DISTINCT`, che ci permette di filtrare i valori **univoci** per una certa colonna. Se ad esempio scriviamo `SELECT DISTINCT(Nome)`, nella tabella risultante non avremo mai due persone con lo stesso nome.

7.2.2 Interrogazione su più tabelle

Come abbiamo già visto con gli operatori dell'algebra relazionale, è possibile effettuare il *prodotto cartesiano* tra due o più relazioni per ottenere tutte le possibili combinazioni di tuple tra queste. In particolare, abbiamo utilizzato il prodotto cartesiano per farci strada verso l'operazione di **join**, che ci permette ci collegare due o più relazioni in base ad attributi all'interno di queste che condividono lo stesso dominio (e, per ottenere risultati coerenti, anche lo stesso *scope informativo*). Nell'algebra relazionale ciò era molto semplice:

- In caso gli attributi delle relazioni coinvolte abbiano lo stesso nome, è sufficiente applicare l'operatore di **Natural Join** ($\bowtie$) tra le due relazioni. Il risultato sarà una selezione delle righe del prodotto cartesiano in cui i due attributi con dominio comune hanno valore uguale.
- In caso i due attributi non abbiano lo stesso nome, si può procedere con la ridenominazione di questi oppure con l'operazione di **Theta Join**, che è quella più comunemente utilizzata nel linguaggio SQL. Tale operazione ci permette di esprimere esplicitamente i due attributi interessati e l'operatore di confronto tra questi.

Osservazione 7.2.3: Prodotto cartesiano in SQL

E' molto importante sapere che se nella clausola `FROM` di un'interrogazione coinvolgiamo più tabelle e non esplicitiamo alcun controllo o alcuna intenzione di effettuare una Join, il linguaggio SQL effettuerà il prodotto cartesiano tra quelle due tabelle.

Nel linguaggio SQL, ottenere queste due operazioni è altrettanto semplice, e dunque si presenta di seguito la sintassi.

Definizione 7.2.1: Natural Join in SQL

Si ottiene inserendo tra una coppia di tabelle nella clausola `FROM` dell'interrogazione la clausola `NATURAL JOIN`. Ad esempio:

```
1  SELECT *
2  FROM Studenti NATURAL JOIN Corsi;
```

Analogamente, si può ottenere anche non utilizzando la clausola `NATURAL JOIN` ma esplicitando il controllo di uguaglianza sugli attributi nella clausola `WHERE`.

```
SELECT *
FROM Studenti, Corsi
WHERE Studenti.CodCorso = Corsi.CodCorso;
```

Questa soluzione è completamente uguale, solo che non stiamo utilizzando il costrutto nativo del linguaggio ma stiamo esplicitando l'operazione da eseguire. Risulta in qualche modo più simile ad una Theta Join.
Nota inoltre che è necessario in questo caso anteporre il nome della tabella all'attributo per evitare conflitti di nome.

Definizione 7.2.2: Theta Join in SQL

L'operazione di Theta Join prevede invece la scrittura manuale della condizione, come visto nell'esempio precedente della Natural Join.

```
SELECT *
FROM Studenti, Corsi
WHERE Studenti.CorsoFrequentato = Corso.CodCorso;
```

In questo caso stiamo esplicitando noi stessi il controllo. Per questo esempio non sarebbe stato necessario anteporre il nome delle tabelle, ma si è deciso di farlo per indicare al lettore quale attributo proviene da quale tabella.
A partire dallo standard SQL-92 è stato introdotto un ulteriore modo per effettuare la Theta Join, definendo una clausola simile a quella per il Natural Join:

```
SELECT *
FROM Studenti JOIN Corsi ON CorsoFrequentato =
   CodCorso;
```

Come si può vedere, la clausola prende la seguente sintassi `Tabella1 JOIN Tabella2 ON CondizioneDiJoin`. Questa particolare sintassi permette di mantenere le condizioni di join separate da eventuali condizioni filtranti il risultato dell'interrogazione.

7.2.3 Ordinamento del risultato

Fatta un'interrogazione, otterremo come risultato di fatto una nuova tabella (come nelle operazioni dell'algebra relazionale ottenevamo una nuova relazione) che conterrà gli attributi richiesti nella `SELECT`, presi dalle tabelle coinvolte nella `FROM` dalle tuple che rispettavano la condizione scritta nella `WHERE`.
Potremmo però decidere voler ordinare i risultati di questa tabella in ordine crescente o decrescente in base ad un determinato attributo. A proposito, SQL ci mette a disposizione la clausola `ORDER BY Attributo1 [ASC/DESC], Attributo2 [ASC/DESC], ....`
Il funzionamento di questa clausola è il seguente, e opera secondo il seguente flusso:

1. Inizialmente, confronta due tuple della tabella secondo il rispettivo valore dell'attributo `Attributo1`.
   - Se si riesce a stabilire un confronto netto (>, <) allora si procede alla seguente coppia di tuple.
   - Altrimenti, si procede a cascata, confrontando le tuple secondo il valore dell'attributo `Attributo2` e così via...
2. Il confronto fra le tuple viene fatto in base a cosa è stato specificato come ordinamento per quelle tuple: se abbiamo scritto `ASC` vogliamo ordinare le tuple in modo **crescente** per quell'attributo (numeri: da 0 -> 9, lettere: da A -> Z); scrivendo `DESC` stiamo invece operando un ordinamento decrescente (numeri: da 9 -> 0, lettere: da Z -> A).

7.2.4 Operatori aggregati

Una vasta gamma di operatori messi a disposizione dal linguaggio SQL rispetto a quelli dell'algebra relazionale sono i cosiddetti **operatori aggregati**. Come suggerisce il nome, si tratta di una famiglia di operatori che agiscono su più tuple contemporaneamente.
Ad esempio, se ci venisse chiesto di determinare **quante** tuple soddisfano una determinata condizione, con i costrutti dell'algebra relazionale ciò sarebbe impossibile: non abbiamo operatori dediti a ciò. Questa, e tante altre operazioni, vengono implementate nel linguaggio SQL per soddisfare richieste quotidiane.

Proposizione 7.2.1: Esecuzione di query con operatori aggregati

Gli operatori aggregati vengono gestiti come un'estensione delle normali interrogazioni. Prima vengono eseguite solo le parti `FROM` e `WHERE`, per ottenere la relazione risultante, e solo in seguito su tale relazione viene applicato l'operatore di aggregazione. Sarà più chiaro con i seguenti esempi, ma si è ritenuto giusto fare una tale premessa.

COUNT

L'operatore `COUNT` serve, intuitivamente, a contare il numero di righe della relazione su cui sono state applicate eventuali restrizioni date dalla clausola `WHERE`.
Assieme al `COUNT` è possibile specificare alcune parole chiave in modo da ottenere risultati diversi:

- `COUNT(DISTINCT ListaAttributi)` ci permette di contare tutte le occorrenze **diverse** dei valori per gli attributi specificati in `ListaAttributi`.
- `COUNT(ALL ListaAttributi)` conta tutte le occorrenze per i valori in `ListaAttributi`, considerando valori duplicati ma **non valori nulli**.
- `COUNT(*)` l'opzione base, ci permette di contare tutte le righe della relazione risultante con duplicati e valori nulli.

Osservazione 7.2.4: Varianti di `COUNT`

Se si specifica unicamente `COUNT(ListaAttributi)`, si assume come keyword di default `ALL`, dunque non si considerano valori nulli.

Esempio 7.2.3: Esempio di query con `COUNT()`

Estrarre il numero di impiegati che appartengono al dipartimento di Produzione.

```
SELECT COUNT(*) AS NumeroImpiegati
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

SUM

L'operatore `SUM(Attributo)` si occupa di restituire la somma di tutti i valori **non nulli** per l'attributo specificato.

Esempio 7.2.4: Esempio di query con `SUM()`

Estrarre la somma degli stipendi degli impiegati del dipartimento di Produzione

```
SELECT SUM(Stipendio) AS SommaStipendi
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

MAX e MIN

Con gli operatori `MAX(Attributo)` e `MIN(Attributo)` il linguaggio SQL ci restituisce rispettivamente il valore massimo e minimo di tutti i valori della relazione per l'attributo specificato. Importante nota è che questi operatori funzionano anche per attributi di tipo testuale, dunque se usati su dei `CHAR` o `VARCHAR` ci restituiranno l'ultima/prima stringa basandosi sull'ordine alfabetico.

Esempio 7.2.5: Esempio di query con `MIN()`

Estrarre lo stipendio minimo degli impiegati del dipartimento di Produzione

```
SELECT MIN(Stipendio) AS StipendioMinimo
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

Esempio 7.2.6: Esempio di query con `MAX()`

Estrarre lo stipendio massimo degli impiegati del dipartimento di Produzione

```
SELECT MAX(Stipendio) AS StipendioMassimo
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

AVG

Infine, l'operatore `AVG(Attributo)` ci permette di ottenere la **media aritmetica** dei valori presenti per l'attributo specificato. Come si può immaginare, questo operatore esegue inizialmente una `SUM` dei valori di quell'attributo, e poi divide tale risultato per il `COUNT` di tali righe.

Esempio 7.2.7: Esempio di query con `AVG()`

Estrarre lo stipendio medio degli impiegati del dipartimento di Produzione

```
SELECT AVG(Stipendio) AS StipendioMedio
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

Osservazione 7.2.5: Ridenominazione con `AS`

In tutti i precedenti esempi si è ridenominato il campo relativo all'operatore aggregato mediante la clausola `AS`. Tale scelta **non è obbligatoria**, ma si è soliti farlo per generare maggiore chiarezza nei confronti di chi legge la nostra interrogazione.

Osservazione 7.2.6: Interrogazioni errate con operatori di aggregazione

Data la seguente interrogazione scritta in linguaggio SQL

```
SELECT Nome, MAX(Stipendio) AS StipendioMassimo
FROM Impiegati
WHERE Dipartimento = "Produzione";
```

si noti come tale interrogazione produca un risultato **errato**, o molto probabilmente **non voluto**.
Si potrebbe pensare che l'operazione restituisca il nome e cognome dell'impiegato con stipendio massimo, bensì il risultato sarà *una relazione in cui per ogni riga verrà associato il nome di un impiegato e lo stipendio massimo del suo dipartimento*. E' dunque *molto probabile* che a fianco di molti impiegati non vi sia scritto il loro vero stipendio bensì quello massimo, calcolato infatti in modo aggregato rispetto alla relazione ottenuta dalla query senza aggregazione.

`GROUP BY`

Per risolvere problemi di natura simile a quelli presentati nell'osservazione precedente, il linguaggio SQL ci mette a disposizione una potentissima clausola, utile ad applicare l'operatore aggregato **separatamente a sottoinsiemi di righe**. Questo è possibile grazie alla clausola `GROUP BY ListaAttributi`.
Tale operatore ci permette di *dividere la tabella in sottoinsiemi, raggruppati in base agli stessi valori per gli attributi specificati in* `ListaAttributi`, e su ogni sottoinsieme *eseguire l'operazione di aggregazione specificata*. Nuovamente, ciò sarà più chiaro con l'esempio di seguito, tuttavia si noti come la suddivisione della tabella in base agli attributi specificati crei una **partizione della tabella di partenza**.

Esempio 7.2.8: Uso dell'operatore `GROUP BY`

Supponiamo di avere la seguente tabella `Impiegati`.

Cognome	Dipartimento	Stipendio

Rossi	Vendite	1800
Bianchi	Produzione	1500
Ferrari	Marketing	1900
Esposito	Vendite	1750
Ricci	Produzione	1600
Marino	Marketing	2100
Greco	Produzione	1550

E di voler ricavare lo stipendio medio per ogni dipartimento.
Dobbiamo chiaramente raggruppare per ogni singolo dipartimento presente e poi calcolare la media per quel dipartimento. La query che risolve tale problema è la seguente:

```
SELECT Dipartimento, AVG(Stipendio)
FROM Impiegati
GROUP BY Dipartimento;
```

Il funzionamento dell'operatore `GROUP BY` si articola dunque nei seguenti passaggi:

1. Viene eseguita la query **senza operazioni di aggregazione**.
2. Si raggruppa (partiziona) la tabella risultante in base agli attributi specificati nella clausola `GROUP BY`. Nel pratico, ciò viene fatto scansionando ogni riga della tabella e accorpando nello stesso gruppo righe che hanno gli stessi valori per gli attributi scritti.
3. Si esegue l'operatore di aggregazione su ogni raggruppamento effettuato.

`HAVING`

Parliamo infine dell'ultimo operatore: `HAVING`. Fondamentale per quando si vogliono **effettuare controlli riguardo a valori prodotti da operatori di aggregazione**.
Come abbiamo già visto, infatti, tali operatori di aggregazione vengono eseguiti solo *dopo* la clausola `WHERE`, e dunque scrivere lì un controllo non avrebbe senso. E' stata predisposta la clausola `HAVING` che risolve proprio questo problema.

Esempio 7.2.9: Esempio di uso di HAVING

Estrarre il nome dei dipartimenti che hanno stipendio medio superiore a 2000.

```
SELECT Dipartimento, AVG(Stipendio)
FROM Impiegati
GROUP BY Dipartimento
HAVING AVG(Stipendio) > 2000;
```

Avendo presentato tutte le clausole fondamentali di una query, si lascia di seguito la generica struttura compatibile con la sintassi del linguaggio SQL:

```
SELECT ListaAttributi
FROM ListaTabelle
[ WHERE Condizioni ]
[ GROUP BY ListaAttributiRaggruppamento ]
[ HAVING CondizioneSuAggregazione ]
[ ORDER BY ListaAttributiOrdinamento ]
```

7.2.5 Operatori insiemistici

Come per l'algebra relazionale, anche il linguaggio SQL mette a disposizione le operazioni di unione, differenza ed intersezione tra più tabelle. Le vediamo di seguito più nel dettaglio.

Osservazione 7.2.7: Compatibilità delle operazioni

Il requisito affinché le operazioni sopra citate siano ben definite è lo stesso dell'algebra relazionale: le tabelle devono essere **union compatibili**.

Non verrà spiegato nuovamente il significato delle varie operazioni, bensì verranno forniti semplicemente degli esempi su tabelle concrete.

`UNION`

Esempio 7.2.10: Esempio di uso di `UNION`

Estrarre i nomi e i cognomi degli impiegati

```
SELECT Nome
FROM Impiegati
UNION
SELECT Cognome
FROM Impiegati;
```

Nota che l'operazione di unione sarebbe possibile anche se stessimo unendo, ad esempio, una tabella che ha un campo `Nome`, definito come stringa, con un campo `TitoloLibro`, sempre definito come stringa. Le due tabelle **sono union compatibili**, ma il risultato *manca di senso*.

`EXCEPT`

La clausola `EXCEPT` corrisponde alla *differenza insiemistica*.

Esempio 7.2.11: Esempio di uso di `EXCEPT`

Estrarre i nomi degli impiegati che non sono anche cognome di qualcun altro.

```
SELECT Nome
FROM Impiegato
EXCEPT
SELECT Cognome
```

```
5       FROM Impiegato;
```

`INTERSECT`

Esempio 7.2.12: Esempio di uso di `INTERSECT`

Estrarre i nomi degli impiegati che sono anche cognome di qualcun altro

```
1       SELECT Nome
2       FROM Impiegato
3       INTERSECT
4       SELECT Cognome
5       FROM Impiegato;
```

7.2.6 Interrogazioni nidificate

Finora abbiamo visto solo interrogazioni in cui le condizioni espresse nella clausola `WHERE` erano relativamente semplici. Tuttavia, in caso di interrogazioni più complesse, è possibile anche scrivere intere *sotto-interrogazioni* il cui risultato verrà usato come valore di confronto nella clausola `WHERE`: si parla in questo caso di **interrogazioni nidificate**.
Supponiamo di voler *estrarre i dati degli impiegati che hanno completato almeno un progetto*. Senza entrare nei dettagli delle singole tabelle, supponiamo di avere le tabelle `Impiegati`, `Progetti`, `ImpiegatiProgetti`. Seppur questo esempio è risolvibile anche in altri modi, si presenta quello utilizzante interrogazioni nidificate.
Innanzitutto bisogna chiedersi: cosa significa che ha completato almeno un progetto? Significa che *è presente nella tabella `ImpiegatiProgetti`*. E come facciamo a controllare la presenza insiemistica di una tupla in una tabella? Il linguaggio SQL ci mette a disposizione vari strumenti.

La clausola `ANY`

La clausola `ANY` ci permette di confrontare un valore in modo che rispetti la condizione con **almeno uno** dei valori risultanti dall'interrogazione nidificata. Intuitivamente, si può vedere come quest'operatore assomigli all'operatore di **appartenenza insiemistica**. Difatti, per risolvere l'interrogazione presentata sopra, è possibile operare nel seguente modo:

```
1       SELECT Cognome, Stipendio
2       FROM Impiegati
3       WHERE Cognome = ANY (
```

```
    SELECT Cognome
    FROM ImpiegatiProgetti
);
```

Come vediamo, nella `WHERE` non stiamo facendo un controllo tra un attributo della tabella `Impiegati` e un valore singolo, bensì dal risultato di una sotto-query.
L'ordine di esecuzione delle operazioni è il seguente:

1. Viene eseguita la query più esterna, senza prestare attenzione alla clausola `WHERE`. In questo modo abbiamo il cognome dell'impiegato, che sarà l'attributo da controllare.
2. Avendo adesso il valore dell'attributo su cui operare il controllo, viene eseguita la query interna, che produrrà una lista di valori.
3. Si effettua il controllo tra il valore nella query esterna con la lista di valori prodotti dalla query interna.
4. Si continua così per tutte le righe degli impiegati...

Un modo alternativo per implementare questa logica è l'utilizzo dell'operatore `IN`, che può essere direttamente sostituito a `= ANY` in quanto significa la stessa cosa (forse è anche più intuitivo, in quanto significa letteralmente "*contenere*").

La clausola ALL

Se invece ci venisse chiesto: *estrarre i dati di tutti gli impiegati che non hanno completato alcun progetto*. In questo caso, non possiamo cavarcela né con una join ne con una condizione semplice. Il fatto che un impiegato non abbia completato alcun progetto vuol dire che tale impiegato **non è presente** nella tabella `ImpiegatiProgetti`. Dobbiamo dunque verificare una *non appartenenza insiemistica*. L'interrogazione presentata si può dunque risolvere come segue:

```
SELECT Cognome, Stipendio
FROM Impiegati
WHERE Cognome != ALL (
    SELECT Cognome
    FROM ImpiegatiProgetti
);
```

In questo caso stiamo usando la clausola `ALL`, che confronta l'attributo specificato con **tutti** gli elementi della lista prodotta come risultato. Restituisce **True** sse. la condizione è verificata per tutte le occorrenze; **False** se anche solo per un elemento della lista di confronto tale condizione non viene rispettata.

Nel nostro caso, per esprimere una *non appartenenza* insiemistica, stiamo chiedendo che un cognome sia diverso da tutti i cognomi che hanno completato progetti. E' un po' come la differenza, in un certo senso.
Anche qui abbiamo un'alternativa sintattica più intuitiva: la clausola `NOT IN`, che sostituisce perfettamente `!= ALL` controllando se tale valore di attributo non è presente nella lista di confronto.

7.3 Il linguaggio di manipolazione dei dati: DML

Andiamo adesso a vedere tutti i comandi relativi al sotto-linguaggio di SQL che ci permette di inserire, modificare ed eliminare valori all'interno delle nostre tabelle.

7.3.1 L'inserimento di dati

L'inserimento di dati all'interno di una relazione viene effettuato attraverso la clausola `INSERT`, che prevede la seguente sintassi:

```
1  INSERT INTO NomeTabella (ListaAttributi)
2  VALUES (ListaValori);
```

Nonostante la sintassi sia abbastanza esplicativa, è giusto fare delle osservazioni:

- Nella `ListaAttributi` specificata dobbiamo inserire il nome degli attributi (le colonne) della tabella nella quale vogliamo inserire i dati. Questi **non devono necessariamente essere tutti gli attributi della tabella**.
- In caso venga omesso uno degli attributi definiti nella creazione della tabella, ci sono due possibilità riguardo al valore che il sistema assegnerà alle colonne mancanti:
  1. Se nella creazione della tabella è stato definito un valore di `DEFAULT` per quel campo, verrà impostato quel valore.
  2. Se invece non è stato definito alcun `DEFAULT`, il sistema imposterà di base `NULL`.
- Nonostante gli attributi in `ListaAttributi` non devono essere necessariamente tutti quelli della tabella originale e **non conti l'ordine**, è invece fondamentale mantenere l'ordine quando si scrivono i valori in `ListaValori`. Dunque ad un attributo attributo_i deve corrispondere un valore_i, che deve essere compatibile di dominio e significato all'attributo per il quale viene valorizzato.

Una variante all'operazione di inserimento consiste nel non specificare la lista di attributi. In questo caso, il linguaggio SQL supporrà che si vorranno inserire tutti gli attributi definiti nello schema della tabella, che dovranno essere dunque inseriti anche nell'ordine di definizione.
Si può anche avere una sintassi del seguente tipo:

```
INSERT INTO NomeTabella (ListaAttributi)
SELECT ListaAttributiCompatibili
FROM ListaTabelle
...
```

All'interno della tabella `NomeTabella` verranno inserite tutte le tuple risultanti dalla query scritta. Questo ovviamente salvo errori dovuti da incompatibilità di domini oppure violazioni dei vincoli di eventuali vincoli di integrità.

Esempio 7.3.1: Esempio di inserimento in una tabella

Vediamo di seguito esempi concreti di inserimento dati all'interno di una tabella con il seguente schema: `Persona(CF, Cognome, DataNascita)`.

```
INSERT INTO Persona (CF, Cognome)
VALUES (RSSMTT, Rossi);
```

Listing 7.4: Inserimento specificando la lista degli attributi.

NOTA: se presente già una tupla in cui `CF = RSSMTT`, l'inserimento restituirà un errore di duplicazione della chiave primaria.

```
INSERT INTO Persona
VALUES (RSSMTT, Rossi, "1980-06-24");
```

Listing 7.5: Inserimento senza specificare la lista degli attributi.

```
INSERT INTO Lavoratori
SELECT CF, Cognome, DataNascita
FROM Persona
WHERE DataNascita < "1990-01-01";
```

Listing 7.6: Inserimento a partire da query.

In questo ultimo inserimento stiamo inserendo nella tabella `Lavoratori` (che assumiamo per semplicità avere lo stesso schema di `Persona`) tutte le persone nate prima del 1990.

7.3.2 La cancellazione dei dati

La cancellazione dei dati da una tabella viene fatto mediante il comando `DELETE`, che segue la seguente sintassi:

```
DELETE FROM NomeTabella
[ WHERE Condizione ]
```

Si tratta, ovviamente, di un comando **molto** delicato, che rischia di commettere errori gravissimi. In particolare, se si omette la condizione di cancellazione (in quanto essa è opzionale) si eliminano tutte le tuple dalla tabella indicata.

Esempio 7.3.2: Esempio di cancellazione da una tabella

Sempre sfruttando la tabella `Persona` dell'esempio precedente, supponiamo di voler eliminare dalla tabella tutte le persone il cui cognome è "Rossi".

```
DELETE FROM Persona
WHERE Cognome = "Rossi";
```

Oppure supponiamo anche di voler fare tabula rasa della tabella: in tal caso sarà molto semplice.

```
DELETE FROM Persona;
```

7.3.3 La modifica dei dati

Per modificare il valore di un attributo in una o più tuple si può usare il costrutto `UPDATE`, che gode della seguente sintassi:

```
UPDATE NomeTabella
SET Attributo = Espressione
[ WHERE Condizione ]
```

Anche qui la condizione è opzionale: se omessa, tutte le tuple della tabella assumeranno il valore specificato nell'espressione.

Osservazione 7.3.1: Utilizzo di query per l'aggiornamento

Come per la clausola `INSERT`, è possibile modificare un valore in una o più tuple in base al risultato di una query. Ovviamente tale query deve **restituire una sola tupla con un solo attributo**, altrimenti la clausola `UPDATE` non saprebbe da quale tupla e/o attributo andare a prendere il nuovo valore. Per completezza, presentiamo la sintassi:

```
UPDATE NomeTabella
SET Attributo = (
  SELECT Attributo
  FROM ListaTabelle
  [ WHERE Condizione ]
)
[ WHERE Condizione ]
```

Osservazione 7.3.2: Aggiornamento dei vincoli di integrità referenziale

Nella definizione dei vincoli di integrità referenziale abbiamo visto due particolari costrutti: `ON UPDATE ...` e `ON DELETE ....` Ovviamente l'esecuzione di tali costrutti entra in gioco ora, con l'esecuzione di una delle due operazioni di modifica o cancellazione.

7.4 Il linguaggio per il controllo degli accessi: DCL

Come abbiamo già detto introducendo il linguaggio SQL è anche possibile, tramite appositi comandi, definire quali utenti hanno quali privilegi (cioè a quali tabelle/singole righe possono accedere).
Un privilegio è formalmente definito da 5 componenti principali:

- **La risorsa a cui si riferisce**, dunque ad esempio la tabella, la vista, il dominio o le singole colonne/righe oggetto dell'autorizzazione.
- **L'utente che concede il privilegio (grantor)**, tipicamente l'admin della base di dati.
- **L'utente che riceve il privilegio (grantee)**.
- **L'azione elementare consentita dal privilegio**.
- **La trasmissibilità del permesso**, ovvero la facoltà o meno del grantee di trasmettere il privilegio che gli è appena stato concesso.

Le azioni concesse sulla risorsa descritta dal privilegio sono:

- `SELECT`, consente la lettura dei dati da quella risorsa.
- `INSERT`, autorizza l'inserimento di tuple nella risorsa.
- `UPDATE`, permette di modificare il contenuto di tuple esistenti.
- `DELETE`, permette di eliminare tuple dalla risorsa.
- `REFERENCES`, consente di definire vincoli di chiavi esterna sulla risorsa.
- `USAGE`, più sottile delle altre opzioni perché consente l'uso di una risorsa che non è necessariamente una tabella (può essere un dominio).

7.4.1 La concessione dei privilegi

La concessione dei privilegi utenti avviene mediante il comando `GRANT`, che rispetta la seguente sintassi:

```
GRANT ListaPrivilegi [ListaColonne]
ON Risorsa
TO ListaUtenti
[ WITH GRANT OPTION ]
```

Analizziamo il funzionamento di questo comando:

1. La `ListaPrivilegi` può contenere una o più delle azioni descritte sopra.
2. Al posto di `ListaColonne` bisogna inserire tutte le colonne della risorsa oggetto della restrizione.
3. In `ListaUtenti` si indica il nome effettivo degli utenti della base di dati a cui si vogliono dare tali privilegi. Un'opzione alternativa è `PUBLIC`, che permette in un colpo solo di attribuire i privilegi a **tutti gli utenti della base di dati**.
4. La clausola `WITH GRANT OPTION` è opzionale e indica la possibilità degli utenti in `ListaUtenti` di passare questo privilegio ad altri utenti sempre mediante il comando `GRANT` con questa sintassi.

Esempio 7.4.1: Esempio di uso di `GRANT`

Forniamo all'utente 'Mario' la possibilità di effettuare interrogazioni sulla sola tabella `Impiegati`

```
GRANT SELECT
```

```
    ON Impiegati
    TO Mario;
```

Forniamo all'utente 'Mario' la possibilità di effettuare interrogazioni sulla sola tabella `Impiegati` e di modificare il solo campo `Stipendio`.

```
GRANT SELECT, UPDATE (Stipendio)
ON Impiegati
TO Mario;
```

Si potrebbero fare tanti, tantissimi altri esempi, ma si lascia al lettore la creatività.

7.4.2 La revoca dei privilegi

La revoca di un privilegio avviene grazie al comando `REVOKE`, che segue la seguente sintassi:

```
REVOKE [ GRANT OPTION FOR ] Privilegi
ON Risorsa
FROM ListaUtenti
[ RESTRICT | CASCADE ]
```

Andiamo a discutere i costrutti univoci di questo comando:

- Specificando `GRANT OPTION FOR` non stiamo revocando agli utenti di `ListaUtenti` i loro privilegi, bensì solo il privilegio di attribuire *quello specifico privilegio descritto di seguito*.
- Similmente a come avevamo fatto per le chiavi esterne, anche qui possiamo specificare cosa deve succedere agli utenti che hanno propagato il loro privilegio ad altri utenti:
  - Con `RESTRICT` ci viene impedito di rimuovere il privilegio ad utenti che l'hanno propagato, impedendoci di lasciare utenti orfani.
  - Con `CASCADE`, viene tolto il privilegio ad ogni utente di `ListaUtenti` e da tutti coloro a cui era stato propagato.

Gli esempi di sintassi sono molto simili a quelli per il comando `GRANT`, e dunque verranno omessi: si faccia riferimento alla sintassi generica descritta sopra.

7.4.3 Le transazioni in SQL

Come già discusso nella Sezione 2.2, le transazioni rappresentano un'unità atomica di istruzioni, che devono portare modifiche consistenti e durevoli alla base di dati.

La definizione di una transazione nel linguaggio SQL avviene grazie a 3 comandi principali:

`BEGIN TRANSACTION`: si occupa di iniziare una transazione. Tutte le operazioni specificate dopo questo comando dovranno essere eseguite di fila, senza interruzioni.

`COMMIT WORK`: si occupa di chiudere la transazione, far eseguire effettivamente le modifiche apportate dai comandi che precedono il `COMMIT` e seguono il `BEGIN` e apportare permanentemente le modifiche alla base di dati.

`ROLLBACK WORK`: annulla tutte le modifiche avvenute sulla base di dati in seguito all'ultimo commit avvenuto da parte di una transazione. Sostanzialmente si usa per riportare la base di dati ad uno stato precedente e consistente, in seguito magari a qualche errore.

Esempio 7.4.2: Uso di transazioni

Supponiamo di voler simulare lo scambio di 1000 euro da un conto corrente ad un altro. Come abbiamo già visto, per evitare *race condition* dobbiamo fare in modo che tutte le operazioni di aggiornamento avvengano in modo contemporaneo.

```
1   BEGIN TRANSACTION;
2
3   UPDATE ContoCorrente
4   SET Saldo = Saldo - 1000
5   WHERE NumeroConto = 12345;
6
7   UPDATE ContoCorrente
8   SET Saldo = Saldo + 1000
9   WHERE NumeroConto = 55555;
10
11  COMMIT WORK;
```

Supponiamo adesso che un utente malintenzionato (?) voglia aggiungere 2000 euro al conto corrente `12345`. Essendo sbadato e forse non a conoscenza delle transazioni, scrive un semplice

```
1   UPDATE ContoCorrente
2   SET Saldo = Saldo + 2000
3   WHERE NumeroConto = 12345;
```

In seguito, gli amministratori della base di dati si renderanno conto dell'errore e decideranno di riportare la situazione ad uno stato corretto. Potranno farlo con un semplice

`ROLLBACK WORK;` che riporterà lo stato della base di dati all'ultimo `COMMIT WORK` incontrato, che nel nostro esempio sarà il completamento del trasferimento dei 1000 euro dai due conti correnti.

8

Dipendenze funzionali

Nel seguente capitolo andremo a definire altri aspetti legati al modello relazionale definito da Edgar F. Codd, in particolare riguardanti alla creazione di particolari vincoli fondamentali per la progettazione di una base di dati corretta.

Definizione 8.0.1: Dipendenza funzionale

Dato uno schema di relazione R e $X, Y \subseteq R$, una **dipendenza funzionale** è una coppia ordinata di X e Y che stabilisce un particolare legame semantico tra i due insiemi. Si denota nel seguente modo

$$X \to Y$$

e si legge **X determina Y**, oppure **Y dipende funzionalmente da X**.
L'insieme X è detto **determinante** o **parte sinistra della dipendenza**, mentre l'insieme Y è detto **dipendente** o **parte destra della dipendenza**.

Per quanto questa definizione matematica possa sembrare complicata, una dipendenza funzionale è un concetto estremamente comune e individuabile in qualsiasi relazione reale che possiamo immaginare. Facciamo un esempio. Supponiamo di avere due automobili dello stesso modello: allora **necessariamente** quelle due automobili non solo avranno lo stesso modello, ma anche la stessa marca. Se prendiamo due automobili di modello *Panda*, è ovvio che la marca dovrà essere *Fiat*, dato che ad oggi non esistono marche diverse che producono lo stesso modello. Ci sarà dunque una dipendenza funzionale del tipo Modello → Marca.

Definizione 8.0.2: Soddisfazione di dipendenza funzionale

Sia r un'istanza di relazione di R (un insieme di tuple).
Diciamo che r **soddisfa** la dipendenza funzionale $X \to Y$ se:

1. Vale che $X, Y \subseteq R$.

2. $\forall t_1, t_2 \in r$, se $t_1[X] = t_2[X]$ allora $t_1[Y] = t_2[Y]$. Questa implicazione vuol dire che presa una qualsiasi coppia di tuple in r, se queste due tuple hanno lo stesso valore per l'insieme degli attributi X **allora** dovranno avere lo stesso valore anche per l'insieme di attributi Y. Questo ovviamente affinché la dipendenza sia soddisfatta. Tuttavia, se le due tuple avessero valori diversi per X, allora quella coppia di tuple soddisferebbe la dipendenza funzionale per la definizione di implicazione.

Due piccole banali osservazioni:

- Un'istanza di relazione vuota soddisfa qualsiasi dipendenza funzionale (non ci sono insiemi di attributi su cui verificare la definizione, dunque è **vacuamente vera**).
- Un'istanza di relazione con una sola tupla soddisfa qualsiasi dipendenza funzionale, in quanto tutti i controlli verranno effettuati sulla tupla stessa.

Osservazione 8.0.1: Natura delle dipendenze funzionali

Come per la scelta degli attributi, delle chiavi, e di tutto ciò che riguarda una base di dati, non esiste una dipendenza funzionale *unica e corretta*: dipende dal contesto della realtà che stiamo esaminando e vogliamo rappresentare.
Per tale ragione, sarebbe errato definire le dipendenze funzionali a partire dalle tuple di una relazione, in quanto le dipendenze funzionali sono a tutti gli effetti dei *vincoli* che vengono definiti sugli attributi, e non sui valori.

Esempio 8.0.1: Esempio di relazioni con dipendenze funzionali

Riprendiamo l'esempio di prima delle automobili e della dipendenza funzionale Modello $\rightarrow$ Marca.
Vediamo di seguito due tabelle, e discutiamo se queste rispettino la dipendenza funzionale scritta sopra.

	MARCA	MODELLO	CILINDRATA	COLORE
t1	FIAT	Tipo	1000	giallo
t2	FIAT	Panda	600	rosso
t3	Citroen	Picasso	1200	blu

Andiamo a verificare se rispetta la dipendenza funzionale partendo dalla definizione, quindi prendendo ogni possibile coppia di tuple.

- Coppia (t1, t2): hanno un valore diverso per l'attributo modello, e dunque la dipendenza è soddisfatta.
- Coppia (t1, t3): hanno un valore diverso per l'attributo modello, e dunque la dipendenza è soddisfatta.
- Coppia (t2, t3): hanno un valore diverso per l'attributo modello, e dunque la dipendenza è soddisfatta.

Ne segue che tale istanza di relazione **rispetta** la dipendenza funzionale Modello → Marca.

	MARCA	MODELLO	CILINDRATA	COLORE
t1	FIAT	Tipo	1000	giallo
t2	Citroen	Tipo	600	rosso
t3	Citroen	Picasso	1200	blu

Andiamo anche qui a verificare ogni coppia di tuple:

- Coppia (t1, t2): hanno lo stesso valore per il modello (t1[Modello] = t2[Modello]) dunque ci aspettiamo che anche il valore di marca sia lo stesso. Non è però il caso, e dunque la coppia **non** rispetta la dipendenza funzionale.

Non c'è neanche bisogno di controllare altre coppie: per la definizione di dipendenza funzionale, questa deve essere soddisfatta **per ogni coppia di tuple**. Dunque se troviamo anche solo **una** coppia per cui questa non vale, possiamo smettere di cercare.

Definizione 8.0.3: Istanza legale

Dato $F = \{X_1 \to Y_1, \ldots, X_n \to Y_n\}$ un insieme di dipendenze funzionali, un'istanza di relazione r si dice **legale** se **soddisfa tutte le dipendenze funzionali di F**.
Ovviamente, una stessa istanza di relazione può essere legale per un certo insieme F e magari non esserlo per un altro.
Come prima, tutte le istanze vuote oppure con una sola tupla sono legali.

Esempio 8.0.2: Esempio di istanze legali

Definiamo $F = \{AB \to C\}$ e consideriamo la seguente istanza di relazione:

	A	B	C	D
t1	a1	b1	c1	d1
t2	a1	b2	c1	d2
t3	a1	b1	c1	d3

Andando ad esaminare ogni coppia di tuple...

- Coppia (t1, t2): hanno valori diversi per la coppia AB, dunque la dipendenza è soddisfatta.
- Coppia (t1, t3): hanno lo stesso valore per la coppia AB. Controlliamo il valore per l'attributo C, che risulta essere uguale e dunque la dipendenza è soddisfatta.
- Coppia (t2, t3): hanno valori diversi per la coppia AB, dunque la dipendenza è soddisfatta.

L'istanza di relazione risulta essere **legale**.
Ma se ridefinissimo $F = \{AB \to C, B \to CD\}$ si può facilmente verificare come l'istanza continui a soddisfare la dipendenza $AB \to C$ ma non soddisfi $B \to CD$, e dunque **non è legale**.

Abbiamo appena visto come un'istanza di relazione sia legale in base ad un insieme F di dipendenze funzionali. Partendo però dall'insieme F potremmo individuare delle dipendenze funzionali "derivate", cioè **non scritte** esplicitamente all'interno di F che però valgono implicitamente da quelle già definite. Questo concetto viene formalizzato nella definizione sottostante.

Definizione 8.0.4: Chiusura di F

Data un'istanza di relazione r legale secondo l'insieme F, definiamo come F^+ l'insieme di **tutte** le dipendenze funzionali soddisfatte da r, sia quelle contenute in F, sia quelle *implicite* o *derivabili* di cui parlavamo prima. Tale insieme F^+ si dice **chiusura di F** e vale ovviamente $F \subseteq F^+$.

L'insieme F^+ è uno strumento potentissimo, che ci permette di ottenere tutti i vincoli relativi ad uno schema di relazione. Ad ora non abbiamo gli strumenti per dimostrare la natura e l'utilità di questo insieme, ma basti sapere che la chiusura di F è **l'insieme da cui traiamo tutti i vincoli per le nostre relazioni**.

Di seguito due proposizioni banali ma essenziali per alcune dimostrazioni che vedremo in seguito.

Proposizione 8.0.1

Dato uno schema di relazione R e un insieme di dipendenze funzionali F, con $X, Y \subseteq R$ sottoinsiemi di attributi:

$$\text{Se } X \to Y \in F \text{ allora } \forall A \in Y, X \to A \in F^+$$

intuitivamente, se A_i sono gli attributi che compongono l'insieme Y, se l'insieme X determina l'intero insieme Y, allora determinerà anche i singoli attributi che lo compongono.

Proposizione 8.0.2

Vale anche il contrario della proposizione soprastante, ovvero:

$$\text{Se } Y = A_1, A_2, \ldots, A_k \text{ e } \forall A_i \in Y, X \to A_i \in F \text{ allora } X \to Y \in F^+$$

Queste due proposizioni ci saranno più utili in seguito, ma molto banalmente ci stanno dicendo che le due scritture riportate di seguito sono equivalenti (cioè hanno lo stesso significato semantico):

$$\{\text{matr} \to \text{nome, cogn}\} \in F^+$$
$$\{\text{matr} \to \text{nome, matr} \to \text{cogn}\} \subseteq F^+$$

Proposizione 8.0.3: Dipendenze funzionali banali

Per quanto sia *banale* dirlo, se si ha una dipendenza del tipo $X \to Y$, con $Y \subseteq X$ questa sarà **sempre soddisfatta**. Da questo il nome **dipendenza funzionale banale**. Intuitivamente, se X determina univocamente Y, essendo Y un sottoinsieme di X è come dire che X determina univocamente sé stesso.

Avendo ora definito molti dei concetti matematici su cui si fonda la teoria relazionale, si vuole fare un appunto su un concetto già visto e stravisto in precedenza, ma mai definito in maniera troppo formale: le chiavi. Ciò che sappiamo delle chiavi è che sono un insieme di attributi che **identificano univocamente ogni tupla** all'interno di una relazione. Notiamo delle analogie con il concetto di dipendenza funzionale appena definito: se le chiavi devono identificare univocamente ogni tupla della relazione, significa che tutti gli attributi sono *funzionalmente dipendenti dalla chiave*. Di seguito la formalizzazione matematica di questo concetto.

Definizione 8.0.5: Chiave

Sia $K \subseteq R$ un sottoinsieme di attributi della relazione. L'insieme K è una chiave di R se rispetta le seguenti proprietà:

1. $K \to R \in F^+$, cioè tutti gli attributi della relazione sono determinati dalla chiave K.
2. Tale sottoinsieme K deve essere **minimale**, cioè **il più piccolo sottoinsieme di attributi per cui vale (1)**. Formalmente scriviamo $\nexists K' \subset K$ t.c. $K' \to R \in F^+$.

Se, preso un insieme di attributi, questo rispetta solo la proprietà (1) allora tale insieme si dice **superchiave**.

8.1 Chiusura di un insieme di dipendenze funzionali

Precedentemente si è accennato alla definizione della chiusura di F, un insieme che ci servirà più avanti per ricavare tutte le dipendenze relative ad uno schema di relazione R. In questa sezione ci muoveremo per fornire le definizioni e dimostrazioni mancanti per raggiungere il nostro obiettivo, e dare una procedura algoritmica per ricavare le chiavi di uno schema e anche le dipendenze funzionali essenziali.
Partiamo introducendo un nuovo insieme, del quale stavolta diamo però una definizione formale.

Definizione 8.1.1: Chiusura con assiomi di Armstrong

Dato F un insieme di dipendenze funzionali, denotiamo con F^A l'insieme che rispetta le seguenti proprietà:

1. $F \subseteq F^A$
2. **Assioma della riflessività**: se $Y \subseteq X$ allora $X \to Y \in F^A$.
3. **Assioma dell'aumento**: se $X \to Y \in F^A$ allora $XZ \to YZ \in F^A, \forall Z \subseteq R$. Questa risulta essere una *definizione ricorsiva* dell'insieme F^A, che ci darà particolari problemi con una dimostrazione successiva.
4. **Assioma della transitività**: se $X \to Y \in F^A$ e $Y \to Z \in F^A$ allora $X \to Z \in F^A$.

Questi appena scritti sono gli assiomi di Armstrong per la definizione dell'insieme F^A. Prima di procedere, introduciamo alcune *regole deduttive*, ovvero delle proprietà di F^A che non sono assiomi in quanto si possono semplicemente dedurre a partire da questi.

5. **Regola dell'unione:** se $X \to Y \in F^A$ e $X \to Z \in F^A$ allora $X \to YZ \in F^A$.
6. **Regola della decomposizione**: se $X \to Y \in F^A$ e $Z \subseteq Y$ allora $X \to Z \in F^A$.

7. **Regola della pseudotransitività**: se $X \to Y \in F^A$ e $WY \to Z \in F^A$ allora $WX \to Z \in F^A$.

Dimostrazione 8.1.1: Regola dell'Unione

Siano $X \to Y \in F^A$ e $X \to Z \in F^A$.

1. Applicando l'assioma dell'**aumento** a $X \to Y$ aggiungendo l'insieme X a entrambi i membri, si ha $XX \to XY$. Poiché stiamo trattando insiemi di attributi, vale l'idempotenza $XX = X$, da cui $X \to XY \in F^A$.
2. Applicando l'assioma dell'**aumento** a $X \to Z$ aggiungendo Y a entrambi i membri, si ottiene $XY \to YZ \in F^A$.
3. Avendo ricavato $X \to XY \in F^A$ e $XY \to YZ \in F^A$, per l'assioma della **transitività** si conclude che $X \to YZ \in F^A$.

Dimostrazione 8.1.2: Regola della Decomposizione

Siano $X \to Y \in F^A$ e $Z \subseteq Y$.

1. Poiché $Z \subseteq Y$, per l'assioma della **riflessività** si ha $Y \to Z \in F^A$.
2. Avendo per ipotesi $X \to Y \in F^A$ e per riflessività $Y \to Z \in F^A$, applicando l'assioma della **transitività** segue direttamente che $X \to Z \in F^A$.

Dimostrazione 8.1.3: Regola della Pseudotransitività

Siano $X \to Y \in F^A$ e $WY \to Z \in F^A$.

1. Applicando l'assioma dell'**aumento** a $X \to Y$ unendo l'insieme di attributi W a entrambi i membri, si ricava $WX \to WY \in F^A$.
2. Poiché $WX \to WY \in F^A$ e per ipotesi $WY \to Z \in F^A$, applicando l'assioma della **transitività** si deduce che $WX \to Z \in F^A$.

Forniamo adesso una dimostrazione che in verità ci sarà più utile in seguito, ma che si ritiene necessario mostrare adesso.

Definizione 8.1.2: Chiusura di un insieme di attributi

Sia R uno schema di relazione e $X \subseteq R$ un sottoinsieme di attributi dello schema. Definiamo la chiusura dell'insieme X nel seguente modo:

$$X_F^+ = \{A | X \to A \in F^A\}$$

ovvero l'**insieme di tutti gli attributi che sono determinati funzionalmente da X**.

Di nuovo, un lemma fondamentale per la teoria relazionale, che ci permette di determinare se una data dipendenza funzionale è rispettata o meno, solo in base alla natura degli insiemi considerati.

Proposizione 8.1.1: Dipendenze e Chiusura di Attributi

Siano R uno schema di relazione ed F un insieme di dipendenze funzionali su R. Si ha che:

$$X \to Y \in F^A \iff Y \subseteq X^+$$

Dimostrazione 8.1.4: Dimostrazione del Lemma

Sia $Y = \{A_1, A_2, \ldots, A_n\}$. Dimostriamo la doppia implicazione.
Parte "se" ($\Leftarrow$). Assumiamo che $Y \subseteq X^+$.

1. Poiché $Y \subseteq X^+$, per ogni $i \in \{1, \ldots, n\}$ si ha che $A_i \in X^+$.
2. Per definizione di chiusura di un insieme di attributi, $A_i \in X^+$ implica che $X \to A_i \in F^A$ per ciascun i.
3. Applicando la regola dell'**unione** alle n dipendenze funzionali $X \to A_i \in F^A$, si ottiene:
$$X \to A_1 A_2 \ldots A_n \in F^A \implies X \to Y \in F^A$$

Parte "solo se" ($\Rightarrow$). Assumiamo che $X \to Y \in F^A$.

1. Poiché $X \to Y \in F^A$ e per ogni $i \in \{1, \ldots, n\}$ vale $\{A_i\} \subseteq Y$, per la regola della **decomposizione** si ha che:
$$X \to A_i \in F^A \quad \forall i \in \{1, \ldots, n\}$$
2. Per definizione di chiusura di un insieme di attributi, il fatto che $X \to A_i \in F^A$ implica che $A_i \in X^+$ per ogni i.
3. Essendo ogni elemento $A_i \in Y$ appartenente a X^+, ne consegue che $Y \subseteq X^+$.